

MEDIATEK

MT8676 Android Audio User Manual

版本: 1.1
出版日期: 2025-07-11

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-08-12	许剑鑫	正式版
1.1	2025-07-11	付帆	新增 1.5 Audio 各模块 Debug 方法介绍

目录

版本记录 2

目录 3

图片目录 4

表格目录 4

1 Audio 5

1.1 概述 5

1.1.1 音频功能简单概述 5

1.1.2 音频功能及框图 5

1.1.3 音频软件架构 13

1.1.4 音频 HAL 14

1.1.5 ALSA 驱动架构概要 17

1.1.6 DAPM 及 DPCM 概述 18

1.2 AAOS 音量控制 19

1.2.1 在 XML 文件中配置音量 20

1.2.2 客制音量命令 21

1.3 外部硬件设备 22

1.3.1 MT8676 I2S 能力支持 22

1.4 联发科 Aurisys 和开放式 DSP 26

1.4.1 概述 26

1.4.2 数据路径客制指南 27

1.4.3 总结 34

1.5 Audio 各模块 Debug 方法介绍 35

1.5.1 Audio Framework Debug 35

1.5.2 Audio HAL Debug 35

1.5.3 ADSP Debug 37

1.5.4 Audio 虚拟化 Debug 38

1.5.5 Tbox Audio Debug 41

1.6 附录 43

1.6.1 MTK MOL 43

附件一 附加条款 44

图片目录

图 1-1. MT8676 音频框图7

图 1-2. MT6368 音频框图7

图 1-3. 音频软件架构14

图 1-4. 音频软件数据流14

图 1-5. 音频 HAL 架构15

图 1-6. 播放架构16

图 1-7. 录制架构17

图 1-8. ALSA PCM 接口18

图 1-9. 音频音量控制20

图 1-10. 音频音量组配置20

图 1-11. 设备增益配置21

图 1-12. Aurisys 概念26

图 1-13. Aurisys 结构27

图 1-14. DTS 属性列表28

图 1-15. memif 定义28

图 1-16. 音频 HAL PCM Config 配置29

图 1-17. ADSP 架构29

图 1-18. 播放音乐的音频路径30

图 1-19. 将 Playback_3 传输到 PMIC30

图 1-20. ADSP 架构31

图 1-21. DSP 任务配置34

图 1-22. Audio HAL enum 值36

图 1-23. ADSP task log 配置37

图 1-24. VAFE binder 线程38

图 1-25. VAFE timer39

图 1-26. VAFE trace 分析39

图 1-27. VADSP android trace40

图 1-28. VADSP yocto trace41

表格目录

表 1-1. GPIO 功能23

表 1-2. I2S 应用25

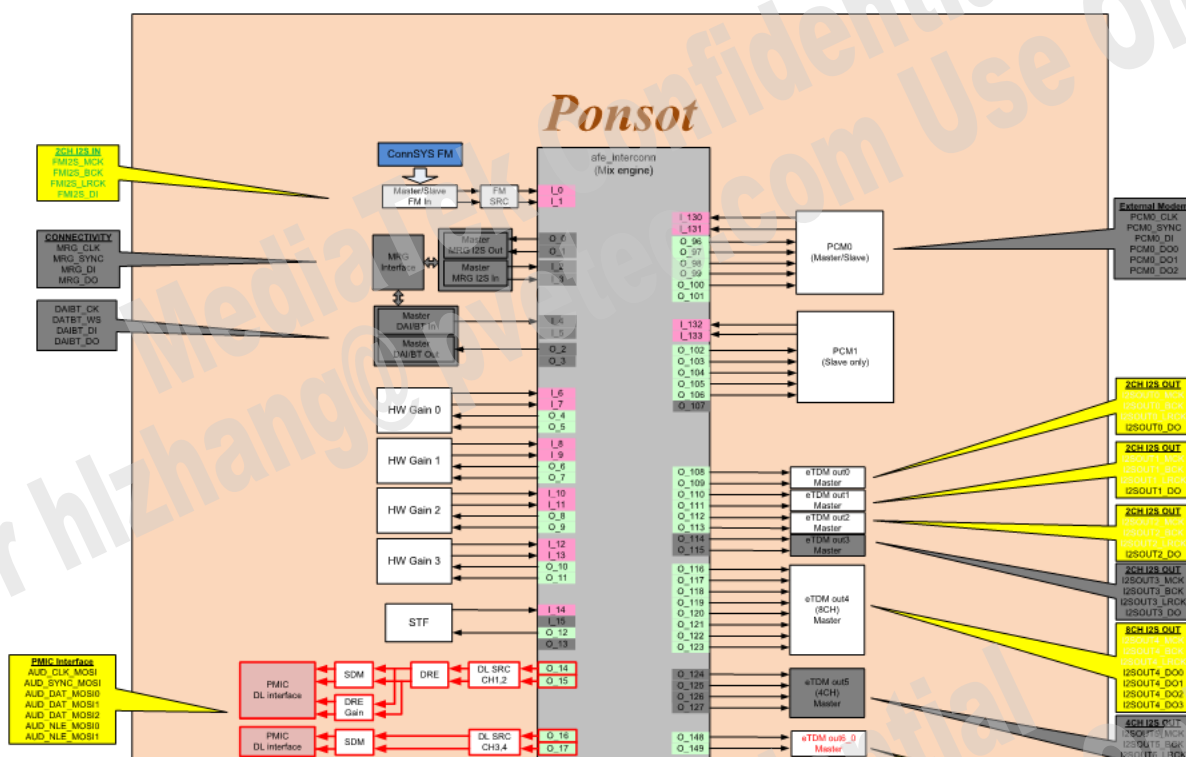
1 Audio

1.1 概述

本文档将介绍音频功能，例如音量控制、音频框架、DSP 音频配置等。首先，我们将概述所支持的功能和音频框图。然后，我们将介绍 AAOS 音量控制及相关配置。最后，我们将介绍 Aurisys 音效框架和 DSP 音频 mix 通路配置。

1.1.1 音频功能简单概述

1.1.2 音频功能及框图





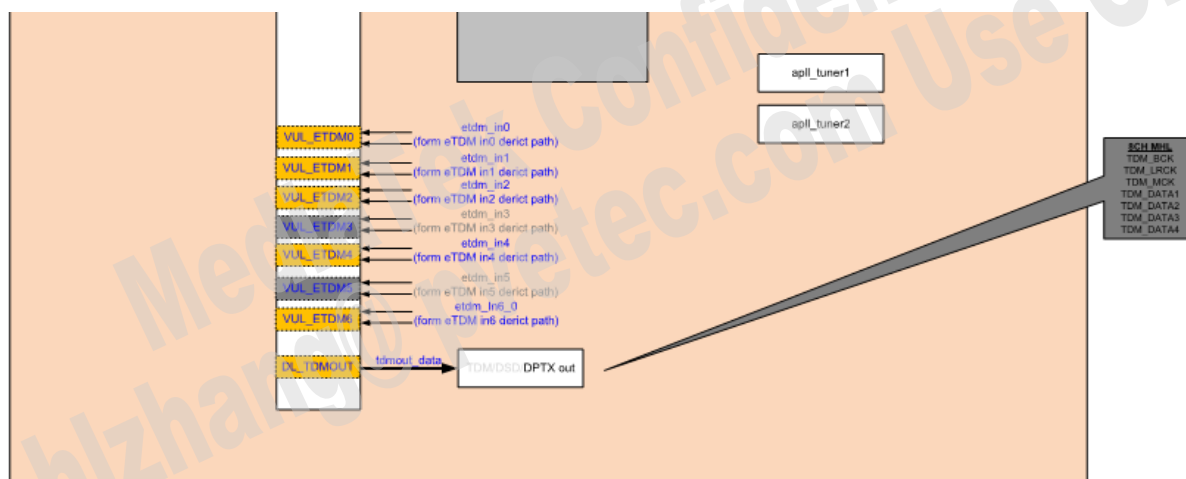


图 1-1. MT8676 音频框图

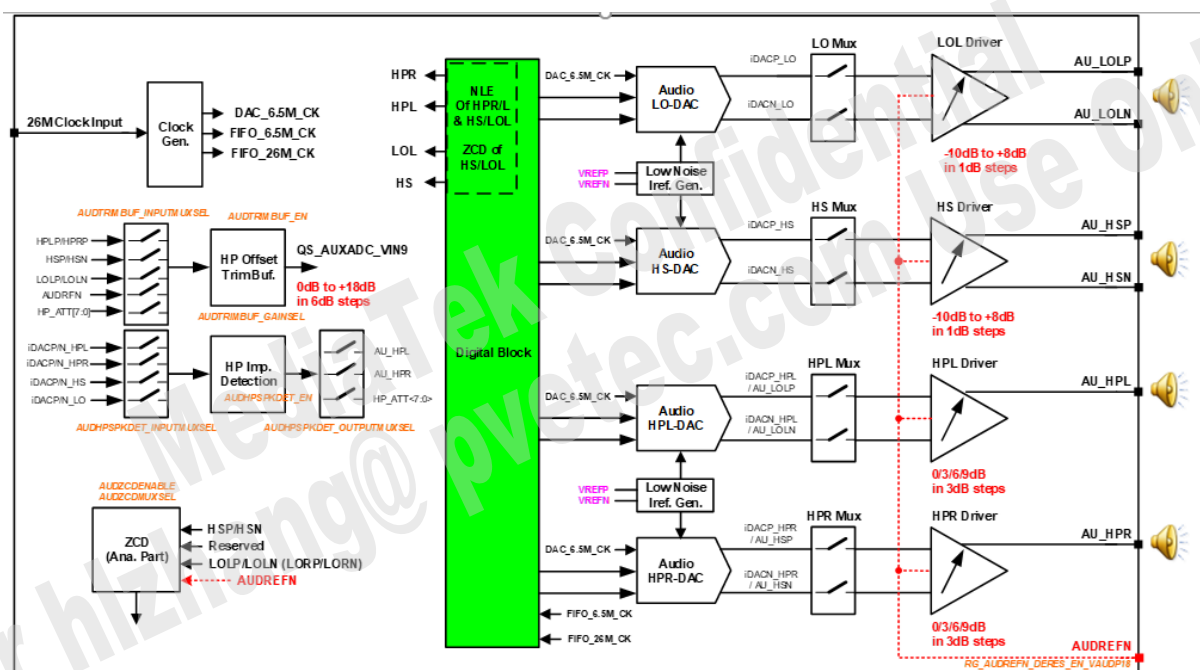


图 1-2. MT6368 音频框图

- MT8676
 - 联发科专有音频接口连接到 PMIC MT6368
 - 使用 MTK PMIC – MT6368 进行音频播放
 - 支持 8kHz, 11.025kHz, 12kHz, 16 kHz, 22.05kHz, 24kHz, 32kHz, 44.1Hz, 48kHz, 96kHz, 192kHz, 384kHz 采样率
 - 使用 MTK PMIC – MT6368 进行音频录制
 - 支持 8kHz, 16kHz, 32kHz, 48kHz, 96kHz, 192kHz 采样率
 - 内部高分辨率、高灵活性的 AFE 互连中的 HW 增益
 - 11 组 Inter IC 音频接口 (I2S)

- 主输出 *5 (其中一个是 8 通道)
- 主输入 *5 (其中一个是 8 通道)
- 从输入(带 SRC)
- 主模式支持 8, 11.025, 12, 16, 22.05, 24, 32, 44.1, 48, 88, 96, 176, and 192kHz 采样率
- 从模式支持 8, 11.025, 12, 16, 22.05, 24, 32, 44.1, 48kHz 采样率
- 支持 16/32 位总线宽度
- Philip 标准和左对齐
- 2 组脉冲编码调制 (PCM)接口
 - Slave PCM for internal Modem*1
 - 支持 16/24 位立体声数据格式
- 2 组增强时分复用 (TDM)接口
 - 主输出 *1 (最多 8 通道)
 - 音频编解码器
 - MP3, AAC, AAC+, AMR-NB, AMR-WB, OGG, WAV, APE
- 音频后处理
 - BesLoudness
 - ACF
 - 第三方支持 (需要客户与第三方联系)
 - Dolby mobile
- MT6368
 - 联发科专有音频接口连接到 MT8676
 - TX 简要规格

MT6897 + MT6368		
DOWNLINK PATH		
# of DACs		3
# of analog outputs		4
Rx concurrency		Up to 2
Max. sampling rate/Bit resolution		192 kHz/24-bit
Output buffer type		DC coupled – ClassAB
Audio buffer	Single-ended (S.E.)/Differential (Diff.)	S.E. *2
	Max. output swing @ 0.1% THD+N	1 Vrms (S.E.) @ 32Ω
	Gain range (dB)	-10~ +8
	Gain per step	1 dB/step
Voice Buffer	Output type	Diff. *1
	Max. output swing @ 1% THD+N	1.87 Vrms-vd @ 32Ω
	Gain range (dB)	-10 ~ +8
	Gain per step	1 dB/step
Line out buffer	Single/Differential	Diff. *1
	Max. output swing	1.88 Vrms-Vd @ 600Ω
SPK amp	Output type	X

– RX 简要规格

MT6897 + MT6368		
UPLINK PATH		
# of ADCs		3
# of analog inputs		4
Tx concurrency		Up to 4
Max sampling rate/Bit resolution		192 kHz/24-bit
Bias circuit		DCC mode
Gain	Gain range (dB)	0~45
	Gain per step	1 dB
# of MICBIAS outputs		3
MICBIAS voltage (V)		1.7 ~ 2.1 & 2.5 ~ 2.7, 0.1per step
MICBIAS current (mA)		Up to 3
# of digital MICs		O (3MICs on MT6368 + 6MICs on MT6897)
Digital MIC CLK Rate (Hz)	26 MHz CLK source	4.33M/3.25M/1.625M/812.5K/406.25K
	32 kHz CLK source	1.625M/812.5K/541.67K
OTHERS		
Accessory detection	UL DCC mode	HP_EINT (MT6368, EINT)
G (Ground) detection		O
Voice wake-up (VoW)		O (1ch or 2ch)
ADSP		O (HiFi3)
SCP		O (RV55NN)

1.1.2.1 音频格式支持

- 播放
 - AAC/HE-AAC v1/HE-AAC v2
 - Android Orientated Support/Not use MediaTek IP
 - 8 kHz~96 kHz; 8 kbps~320 kbps
 - Mono/Stereo support
 - Bitrate Mode: VBR/CBR
 - File Extension: .aac(ADTS, ADIF), .m4a, .mp4, .3gp, .ts
 - Profile: 1) LC, HEAAC V1, V2; 2) LD, ELD
 - LC, 48 kHz, 128 kbps, stereo → MCPS = 11; HEv1, 22.05 kHz, 128 kbps, stereo → MCPS = 31; HEv2, 22.05 kHz, 32 kbps, stereo → MCPS = 40; ELD, 44.1 kHz, 128 kbps MCPS = 13
 - AMR
 - Android Orientated support/Use MediaTek IP
 - 8 kHz, 4.75 kbps~12.2 kbps

- Mono support
- Bitrate mode: CBR
- File Extension: .amr
- 8 kHz, 12.2 kbps, mono -> Mediaserver: 50 MCPS, AMR Dec: 7 MCPS
- AWB
 - Android Orientated support/Use MediaTek IP
 - 16 kHz, 6.6 kbps~23.85 kbps
 - Mono support
 - Bitrate mode: CBR
 - File Extension: .awb
 - 16 kHz, 23.85 kbps, mono -> Mediaserver: 60 MCPS, AMR Dec: 18 MCPS
- MIDI
 - Android Orientated support/Not use MediaTek IP
 - 22.05 kHz
 - Stereo Support
 - File Extension: .mid, .midi, .smf, .rtttl, .xmf, .rtx, .ota, .imy
- MP2
 - Android Not support/Use MediaTek IP
 - 8 kHz ~ 48 kHz; 8 kbps ~ 320 kbps
 - Mono/Stereo support
 - Bitrate Mode: VBR/CBR
 - File Extension: .mp2
 - Profile: MPEG1-Layer2, MPEG2-Layer2, MPEG2.5-Layer2
 - MPEG1, 48 kHz, 256 kbps, stereo → 11 MCPS
- MP3
 - Android Orientated support/Use MediaTek IP
 - 8 kHz ~ 48 kHz; 8 kbps ~ 320 kbps
 - Mono/Stereo support
 - Bitrate Mode: VBR/CBR
 - File Extension: .mp3
 - Profile: MPEG1-Layer3, MPEG2-Layer3, MPEG2.5-Layer3
 - MPEG1, 48 kHz, 256 kbps, stereo → 15 MCPS
- OGG VORBIS
 - Android Orientated support/Use MediaTek IP
 - 8 kHz ~ 192 kHz; 10 kbps ~ 320 kbps
 - Mono/Stereo support
 - Bitrate Mode: VBR
 - File Extension: .ogg, .oga
 - 48 kHz, 250 bps, stereo → MCPS = 25
- WAV (ADPCM)
 - Android Not support/Use MediaTek IP
 - 8 kHz ~ 192 kHz; 4 kbps ~ 192 kbps
 - Bitrate Mode: CBR
 - File Extension: .wav
 - Profile: DVI/MA ADPCM; MS ADPCM

- DVI/MA 48 kHz, stereo → 3.44 MCPS; DVI/MA 192 kHz, stereo → 13.6 MCPS; MS, 48 kHz, stereo → 4.17 MCPS; MS, 192 kHz, stereo → 16.3 MCPS
- WAV (Alaw/Ulaw)
 - Android Orientated support/Use MediaTek IP
 - 6 kHz ~ 96 kHz; 48 kbps ~ 3072 kbps
 - Mono/Stereo support
 - File Extension: .wav
- WAV (Raw)
 - Android Orientated support/Use MediaTek IP
 - 6 kHz~96 kHz; 48 kbps~3072 kbps
 - 1 channel ~ 8 channels support
 - File Extension: .wav
- APE
 - Android Not support/Use MediaTek IP
 - 6 kHz ~ 96 kHz; 29 kbps ~ 836 kbps
 - Mono/Stereo support
 - Bitrate Mode: VBR
 - File Extension: .ape
 - Profile: fast, normal, high, extra high
 - APE normal compress type, 44.1 kHz, 675 kbps, stereo → MCPS = 50
- WMA (Need license)
 - Android Not support/Use MediaTek IP
 - 8 kHz ~ 48 kHz; 5 kbps ~ 320 kbps
 - Mono/Stereo support
 - Bitrate Mode: VBR
 - File Extension: .wma
 - Profile: wma v1; wma v2
 - 32 kHz, 22 kbps, stereo → MCPS = 22.57
- FLAC
 - Android Orientated support/Use MediaTek IP
 - 8 kHz ~ 48 kHz; 87 kbps ~ 396 kbps
 - Mono/Stereo support
 - Bitrate Mode: VBR
 - File Extension: .flac
 - 44.1 kHz, 745 kbps, level5, stereo → MCPS = 10
- 录制
 - AAC
 - Android Orientated support/Not use MediaTek IP
 - 8 kHz ~ 48 kHz; 8 kbps~160 kbps
 - Mono/Stereo support
 - Bitrate Mode: CBR
 - File Extension: .3gp, .aac
 - Profile: Low Complexity; High Efficiency; Enhanced Latency Delay
 - 48 kHz, 128 kbps, stereo (Low Complexity) → MCPS = 32; 48 kHz, 128 kbps, stereo (High Efficiency) → MCPS = 75; 48 kHz, 128 kbps, stereo (Low Latency Delay) → MCPS = 40;

- AMR
 - Android Orientated support/Use MediaTek IP
 - 8 kHz; 4.75 kbps~12.2 kbps
 - Mono support
 - Bitrate Mode: CBR
 - File Extension: .3gp, .amr
 - 8 kHz, 12.2 kbps, mono → MCPS = 39
- AWB
 - Android Orientated support/Use MediaTek IP
 - 16 kHz; 6.60 kbps~23.85 kbps
 - Mono support
 - Bitrate Mode: CBR
 - File Extension: .3gp, .awb
 - 16 kHz, 23.85 kbps, mono → MCPS = 77
- OGG
 - Android Not support/Use MediaTek IP
 - 8 kHz~ 48 kHz; 31.98 kbps~202.96 kbps
 - Mono/Stereo support
 - Bitrate Mode: VBR
 - File Extension: ..ogg
 - 48 kHz, 128 kbps, stereo → MCPS = 40
- ADPCM
 - Android Not support/Use MediaTek IP
 - 8 kHz~ 48 kHz; 4 kbps~192 kbps
 - Mono/Stereo support
 - Bitrate Mode: CBR
 - File Extension: ..wav
 - 16 kHz, stereo, DIV/MA → MCPS = 2.88; 48 kHz, stereo, DIV/MA → MCPS = 8.5; 16 kHz, stereo, MS → MCPS = 5.08; 48 kHz, stereo, MS → MCPS = 14.97

1.1.3 音频软件架构

图 1-3 和图 1-4 分别是音频软件架构和数据流。在这两张图中可以看到软件组件和数据路径。

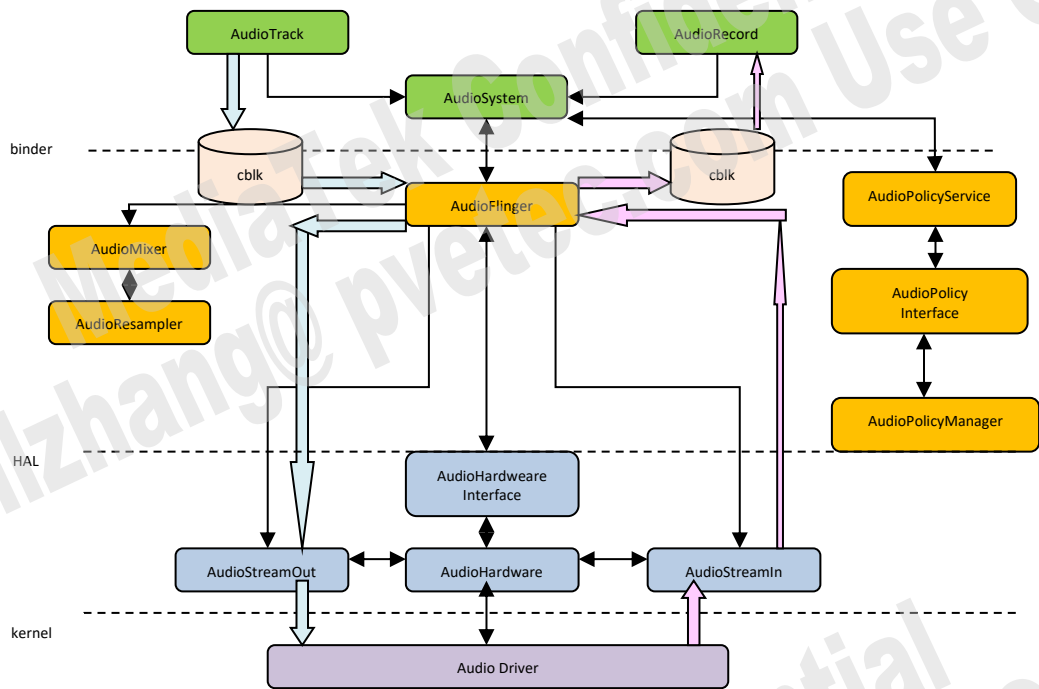


图 1-3. 音频软件架构

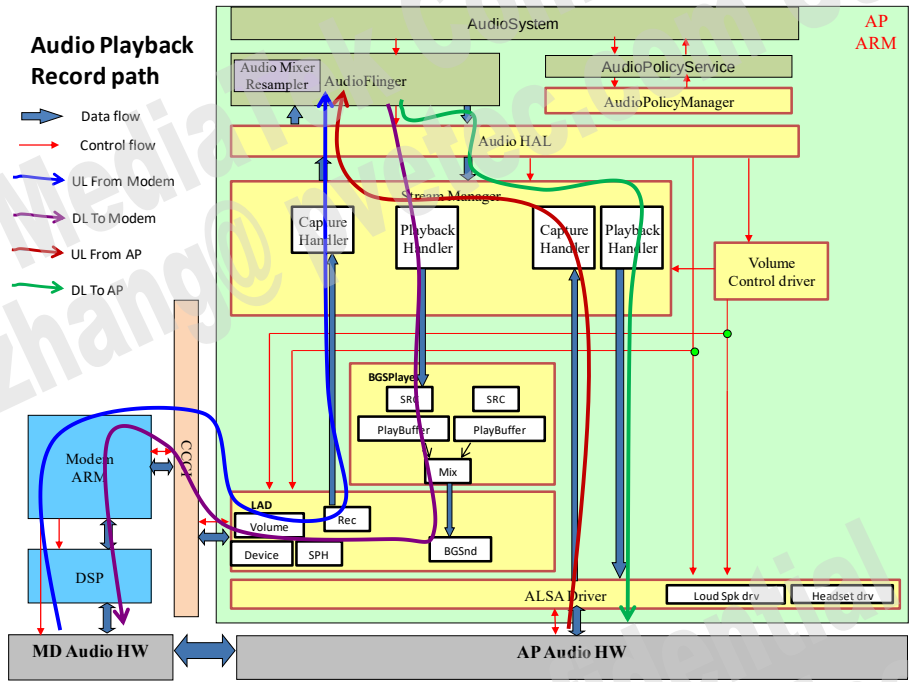


图 1-4. 音频软件数据流

1.1.4 音频 HAL

图 1-5 是音频 HAL 架构。

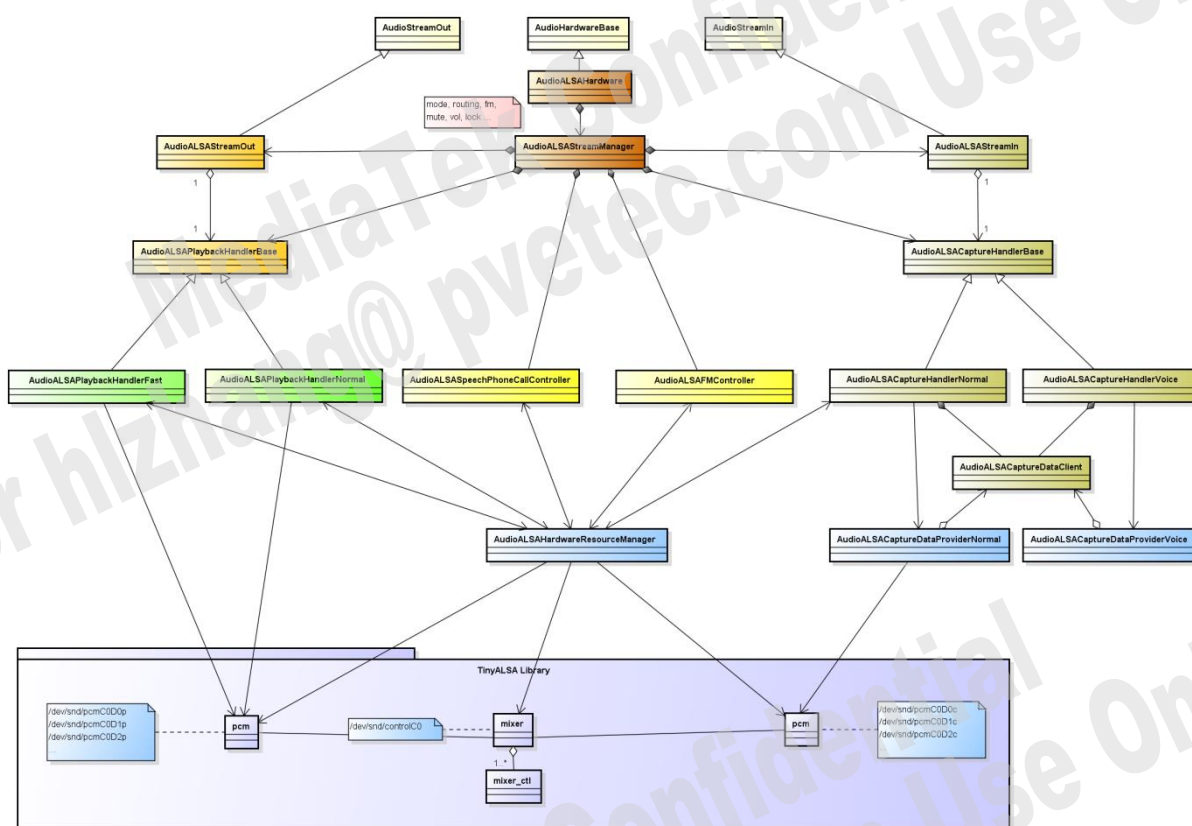


图 1-5. 音频 HAL 架构

HAL 的顶层控制接口是 `AudioALSAHardware`、`AudioALSAStrOut` 和 `AudioALSAStrIn`。这些类都使用 `AudioALSAStrManager` 来控制音频模式、打开/关闭，输入/输出流。因此 `AudioALSAStrManager` 将拥有所有音频环境信息，如模式、路由、音量和静音信息。

1.1.4.1 音频 HAL 播放

AudioALSAStreamOut: 执行写入/待机/路由操作的类

- 不实现细节本身，而是使用 Playback Handler 执行实际打开/关闭/路由/写入操作
- 不同场景使用不同类型的 Playback Handler
- 第一次写入时，Stream Out 将调用 StreamManager 中的 createPlaybackHandler()来获取 Playback Handler 的指针
- 在 standby()时，destroyPlaybackHandler()

不要在 playback handler 中设置硬件寄存器，而是使用 TinyALSA 库来调用

- `pcm_open() / pcm_close()`
 - 控制 AFE 硬件路径和内存设置
- `mixer open()/mixer close()`

- 获取各种类型的混音器控制, 比如 Speaker_Amp_Switch, Voice_Amp_Switch, Audio_Amp_R_Switch, and Audio_Amp_L_Switch, 以控制编解码器驱动程序。
- pcm_write()
 - 将 PCM 数据写入 SRAM/DRAM

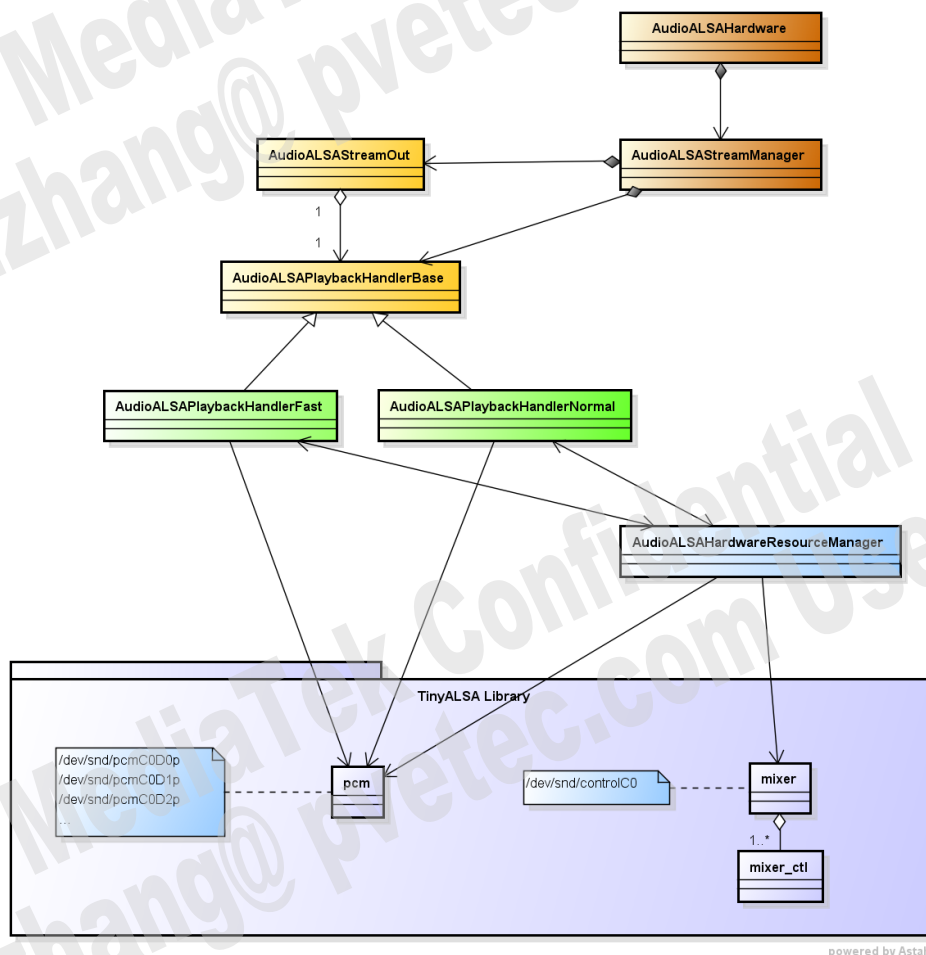


图 1-6. 播放架构

1.1.4.2 音频 HAL 录制

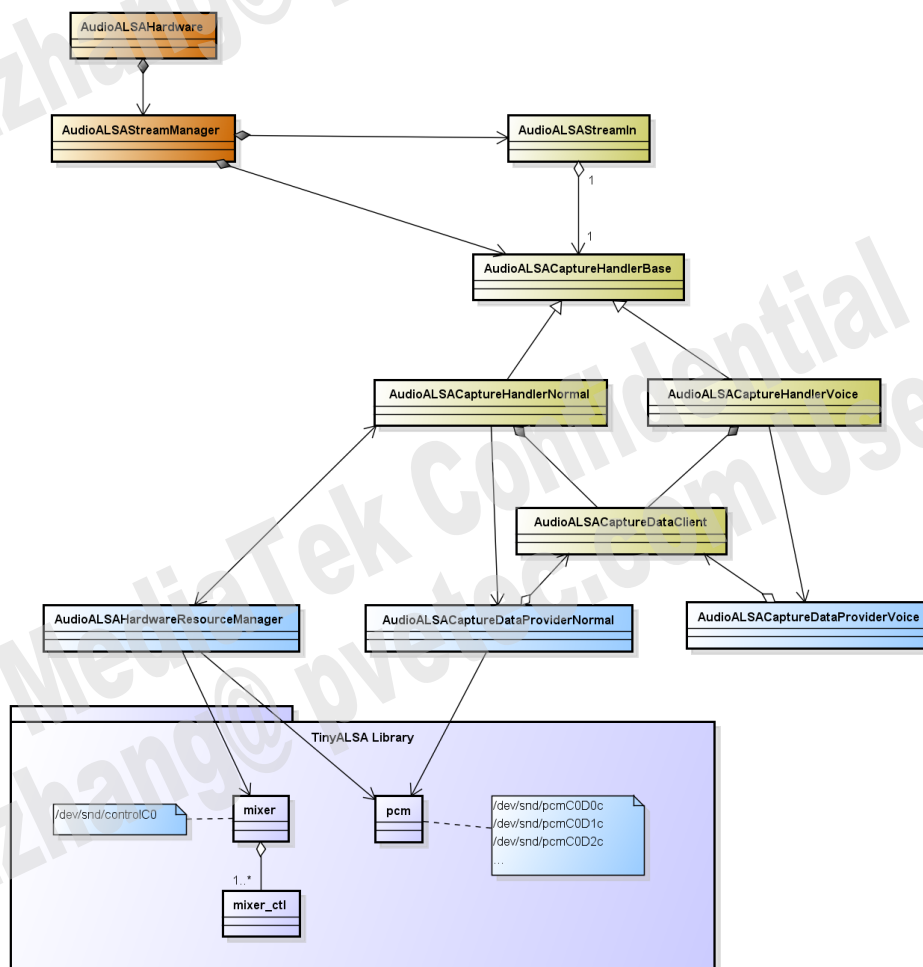
AudioALSASStreamIn：执行读取/待机/路由操作的类

- 不实现细节本身，而是使用 Capture Handler 绕过打开/关闭/路由/读取操作
- 不同场景使用不同类型的 Capture Handler
- Stream In 将在第一次 read()时调用 StreamManager 中的 createCaptureHandler()来获取 Capture Handler 的指针
- 此外，在 standby()时，destroyCaptureHandler()

不要在 Capture Handler 中设置硬件寄存器，而是使用 TinyALSA 库来调用

- pcm_open() /pcm_close()

- 用于控制 AFE 硬件路径、内存设置
- mixer_open()/mixer_close()
 - 用于获取各种类型的混音器控制，如 Audio_ADC_1_Switch、Audio_ADC_2_Switch、Audio_Preamplifier1_Switch、Audio_Preamplifier2_Switch，以控制编解码器驱动程序
- pcm_read()
 - 用于将 PCM 数据读取到 SRAM/DRAM 中



powered by Astah

图 1-7. 录制架构

1.1.5 ALSA 驱动架构概要

高级 Linux 声音架构(ALSA)为 Linux 操作系统提供音频和 MIDI 功能。ALSA 具有以下重要功能：

- 高效支持所有类型的音频接口，从消费级声卡到专业多声道音频接口
- 完全模块化的声音驱动程序

- SMP 和线程安全设计
- 用户空间库 (alsa-lib) 简化应用程序编程并提供更高级别的功能
- 支持较旧的开放声音系统(OSS) API, 为大多数 OSS 程序提供二进制兼容性

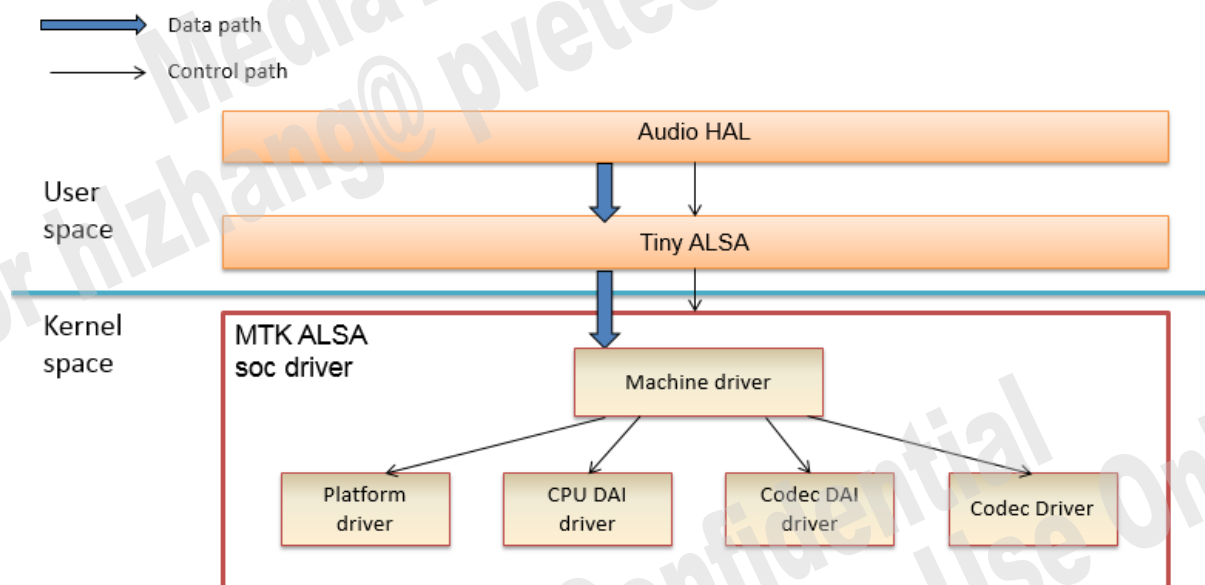


图 1-8. ALSA PCM 接口

1.1.6 DAPM 及 DPCM 概述

动态音频电源管理 (DAPM) 旨在允许便携式 Linux 设备始终使用音频子系统的最小电量。它独立于其他内核 PM, 因此可以轻松与其他 PM 系统共存。

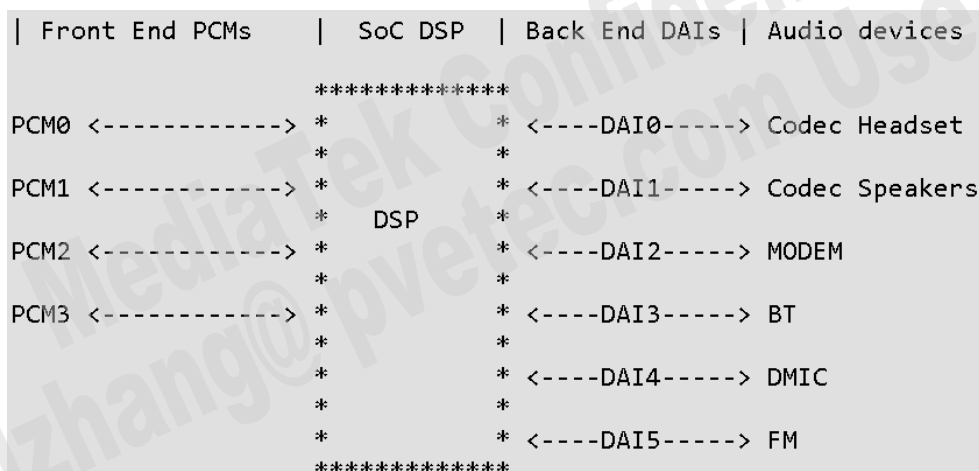
DAPM 对所有用户空间应用程序也完全透明, 因为所有电源切换都在 ASoC 核心内完成。用户空间应用程序无需更改代码或重新编译。DAPM 根据设备内的任何音频流 (捕获/播放) 活动和音频混音器设置做出电源切换决策。

所有 DAPM 电源切换决策都是通过查阅整个机器的音频路由图自动做出的。此图特定于每台机器, 由每个音频组件 (包括内部编解码器组件) 之间的互连组成。以下所有影响电源的音频组件都称为小部件。有关更多详细信息, 请参阅: <https://www.kernel.org/doc/html/v6.1/sound/soc/dpcm.html>。

音频驱动基于标准 DAPM 和 DPCM (动态 PCM) 架构来控制 pcm 的打开/关闭、电源的开/关、时钟的开/关。

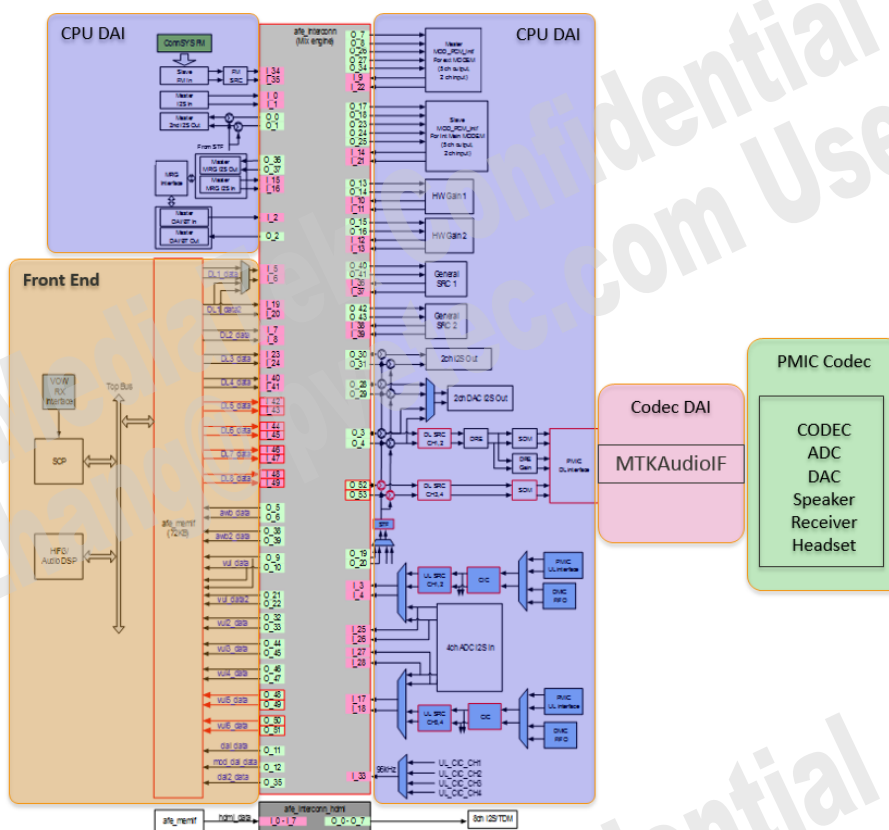
DPCM 将 PCM 分为 Front End/Back End PCM:

- FE PCM: Control of DMA (MEMMEMIF)
- BE PCM: Control of DAI (ADDA, PCM IF, I2S)



更多详细信息可以参考: <https://www.kernel.org/doc/html/v6.1/sound/soc/dpcm.html>。

以下是 MT8676 FE(memif)和 BE(ADDA/I2S/PCM/Hostless/..)框图:



1.2 AAOS 音量控制

根据谷歌的定义，AAOS 的实现是使用硬件功放来控制音量，而不是软件混音器，所以建议使用 smartpa 或者其他硬件增益来调节音量。MTK 在内部 adsp 中提供了调节音量的接口和软件路径，如图 1-9 所示。

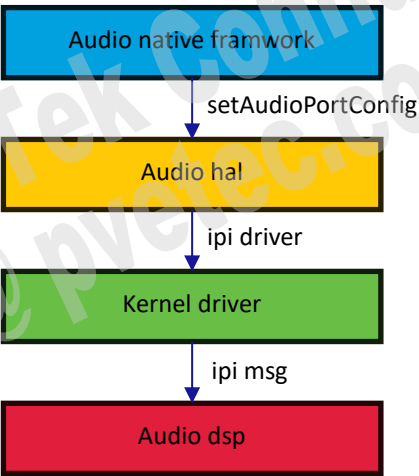


图 1-9.音频音量控制

1.2.1 在 XML 文件中配置音量

要配置音量，我们需要修改 car_audio_configuration.xml 和 audio_policy_configuration.xml。

car_audio_configuration.xml 定义音量组，用于管理音频区域内一组设备的音量。对于每个音量组，音量都可以独立控制。产生的增益在相关设备上配置，以供车辆的放大器应用。每个音量组应包含一个或多个具有相关地址的输出设备。地址应与 audio_policy_configuration.xml 中定义的输出设备相对应。

```
<carAudioConfiguration version="3">
  <zones>
    <zone name="primary zone" isPrimary="true" audioZoneId="0" occupantZoneId="0">
      <zoneConfigs>
        <zoneConfig name="primary zone config" isDefault="true">
          <volumeGroups>
            <group>
              <device address="bus0_media_out">
                <context context="music"/>
              </device>
              <device address="bus3_call_ring_out">
                <context context="call_ring"/>
              </device>
              <device address="bus6_notification_out">
                <context context="notification"/>
              </device>
              <device address="bus7_system_sound_out">
                <context context="system_sound"/>
                <context context="emergency"/>
                <context context="safety"/>
                <context context="vehicle_status"/>
                <context context="announcement"/>
              </device>
            </group>
          </volumeGroups>
        </zoneConfig>
      </zoneConfigs>
    </zone>
  </zones>
</carAudioConfiguration>
```

图 1-10. 音频音量组配置

Audio_policy_configuration.xml 定义了音量组增益。每个音量组都有最小、最大和默认增益值，以及基于 audio_policy_configuration.xml 中为与音量组关联的设备配置的值的步长。

```
<devicePort tagName="bus0_media_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus0_media_out">
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="0" maxValueMB="300" defaultValueMB="300" stepValueMB="30"/>
  </gains>
</devicePort>
<devicePort tagName="bus1_navigation_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus1_navigation_out">
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="0" maxValueMB="300" defaultValueMB="300" stepValueMB="30"/>
  </gains>
</devicePort>
```

图 1-11. 设备增益配置

我们提供最小 0 到最大 300 的增益等级，并且增益将通过对数转换函数映射到浮点值 0~1 并应用于数字音频流。

1.2.2 客制音量命令

音量命令由音频 flinger 通过 setAudioPortConfig 发送，并在 AudioALSAPlaybackHandlerXXX::setVolume 中处理命令。客户可以修改函数以进行自定义，例如调整音量曲线和设置 kcontrol。

1.3 外部硬件设备

1.3.1 MT8676 I2S 能力支持

I2S 能力接口描述

- MTK 音频接口
 - 适用于 MT6368 的 MTK 特定接口
- I2S0 – ADC/DAC 接口
 - 适用于任何音频接口
 - 支持**主输入**的标准 I2S
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位
- I2S1 – ADC/DAC 接口
 - 支持**主输出**的标准 I2S
 - 最多 1 条数据线支持两个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384k/32 位
- I2S2 – ADC/DAC 接口
 - 支持**主输入**的标准 I2S
 - 最多 2 条数据线支持四个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位
- I2S4 – ADC/DAC 接口
 - 支持**主输出**的标准 I2S
 - 最多 4 条数据线支持八个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位
- I2S6 – ADC/DAC 接口
 - 支持**主输入**的标准 I2S
 - 最多 1 条数据线支持两个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位

IP	Pin Name
I2S0	I2SIN0_LRCK / I2SIN0_BCK / I2S0_MCK / I2SIN0_DI/ I2SOUT0_DO
I2S1	I2SIN1_LRCK / I2SIN1_BCK / I2S1_MCK / I2SIN1_DI/ I2SOUT1_DO
I2S2	I2SIN2_LRCK / I2SIN2_BCK / I2SIN2_DI/ I2SOUT2_DO
I2S4	I2SIN4_LRCK / I2SIN4_BCK / I2SIN4_DI0/ I2SOUT4_DO0 / I2SIN4_DI1/ I2SOUT4_DO1 / I2SIN4_DI2/ I2SOUT4_DO2 / I2SIN4_DI3/ I2SOUT4_DO3
I2S6	I2SIN6_LRCK / I2SIN6_BCK / I2SIN6_DI/ I2SOUT6_DO
PMIC DL Interface	AUD_CLK_MOSI AUD_DAT_MOSI0 AUD_DAT_MOSI1
PMIC UL Interface	AUD_CLK_MISO AUD_DAT_MISO0 AUD_DAT_MISO1

- MCLK 支持(I2S0/I2S1/I2S2/I2S4/I2S6)
 - 同步 MCLK 和 I2S BCK
 - 192k 模式下的能力高达 256fs
 - 96k 模式下的能力高达 512fs
 - 48k 模式下的能力高达 1024fs
 - 不支持 192fs 或 384fs 相关

1.3.1.1 I2S 连接应用简介

联发科在 HW 功能 (IP: 知识产权) 和 Pad 之间采用高度灵活的设计。

IP 和 Pad 之间的关系: 可以使用 AUX_FUNC 来选择 Pad 连接到哪个 IP。

表 1-1. GPIO 功能

Aux Func.0	Aux Func.1	Aux Func.2	Aux Func.3	Aux Func.4	Aux Func.5	Aux Func.6	Aux Func.7
B:GPIO15	O:I2SIN0_BCK	-	-	-	I1:ADSP_JTAG0_TRSTN	I0:SCP_JTAG_LITTLE_TRSTN_VCORE	I0:CONN_BGF_MCU_TRST_B
B:GPIO16	O:I2SIN0_LRCK	-	-	-	I0:ADSP_JTAG0_TCK	I1:SCP_JTAG_LITTLE_TCK_VCORE	I0:CONN_BGF_MCU_TCK
B:GPIO17	I0:I2SIN0_DI	-	-	-	I1:ADSP_JTAG0_TMS	I1:SCP_JTAG_LITTLE_TMS_VCORE	I1:CONN_BGF_MCU_TMS
B:GPIO18	O:I2SOUT0_DO	-	-	-	O:ADSP_JTAG0_TDO	O:SCP_JTAG_LITTLE_TDO_VCORE	O:CONN_BGF_MCU_TDO
B:GPIO19	O:I2SIN1_BCK	-	-	-	I1:ADSP_JTAG0_TDI	I1:SCP_JTAG_LITTLE_TDI_VCORE	I0:CONN_BGF_MCU_TDI
B:GPIO20	O:I2SIN1_LRCK	-	I0:MFG_EB_JTAG_TRSTN	-	I1:ADSP_JTAG1_TRSTN	I0:SCP_JTAG0_TRSTN_VCORE	I0:CONN_WF_MCU_TRST_B
B:GPIO21	I0:I2SIN1_DI	-	I0:MFG_EB_JTAG_TCK	-	I0:ADSP_JTAG1_TCK	I1:SCP_JTAG0_TCK_VCORE	I0:CONN_WF_MCU_TCK
B:GPIO22	O:I2SOUT1_DO	-	I0:MFG_EB_JTAG_TMS	-	I1:ADSP_JTAG1_TMS	I1:SCP_JTAG0_TMS_VCORE	I1:CONN_WF_MCU_TMS
B:GPIO23	O:I2SIN2_BCK	O:I2SIN0_MCK	O:MFG_EB_JTAG_TDO	I0:DSI1_TE	O:ADSP_JTAG1_TDO	O:SCP_JTAG0_TDO_VCORE	O:CONN_WF_MCU_TDO
B:GPIO24	O:I2SIN2_LRCK	O:I2SIN1_MCK	I0:MFG_EB_JTAG_TDI	O:LCM1_RST	I1:ADSP_JTAG1_TDI	I1:SCP_JTAG0_TDI_VCORE	I0:CONN_WF_MCU_TDI
B:GPIO25	I0:I2SIN2_DI	O:BPI_BUS20	I0:DSI2_TE	O:CONN_BPI_BUS20	O:ANT_SEL20	-	-
B:GPIO26	O:I2SOUT2_DO	O:BPI_BUS21	O:LCM2_RST	O:CONN_BPI_BUS21	O:ANT_SEL21	-	-
B:GPIO113	O:MIPI4_D_SCLK	I0:DSI1_TE	-	O:DMIC4_CLK	-	O:MD_GPS_L1_BLA_NK	-
B:GPIO114	B0:MIPI4_D_SDATA	O:LCM1_RST	-	I0:DMIC4_DAT	-	O:MD_GPS_L5_BLA_NK	-
B:GPIO109	O:MIPI2_D_SCLK	O:CONN_MIPI2_SCLK	O:DMIC1_CLK	-	-	-	-
B:GPIO110	B0:MIPI2_D_SDATA	B0:CONN_MIPI2_SDATA	I0:DMIC1_DAT	-	-	-	-
B:GPIO107	O:MIPI1_D_SCLK	O:CONN_MIPI1_SCLK	O:DMIC3_CLK	-	-	-	-
B:GPIO108	B0:MIPI1_D_SDATA	B0:CONN_MIPI1_SDATA	I0:DMIC3_DAT	-	-	-	-
B:GPIO63	-	-	O:BPI_BUS22	O:SPI5_B_CLK	O:I2SOUT4_DATA3	O:CONN_BPI_BUS22	-
B:GPIO4	-	-	B0:SPI7_A_MI	O:SPI5_B_CS	O:I2SIN4_LRCK	O:CONN_TCXOENA_REQ	O:DBG_MON_A20
B:GPIO62	-	-	B0:SPI7_A_MO	-	O:I2SOUT4_DATA0	O:WIFI_TXD	O:DBG_MON_A21
B:GPIO137	B1:SCL0	-	O:SPI7_A_CLK	-	O:I2SOUT4_DATA1	O:MD32_0_GPIO0	O:DBG_MON_A22
B:GPIO138	B1:SDA0	-	O:SPI7_A_CS	-	O:I2SOUT4_DATA2	O:MD32_1_GPIO0	O:DBG_MON_A23
B:GPIO65	-	O:CMFLASH0	-	O:USB_DRVVBUS	O:I2SIN4_BCK	-	O:DBG_MON_A24

Aux Func.0	Aux Func.1	Aux Func.2	Aux Func.3	Aux Func.4	Aux Func.5	Aux Func.6	Aux Func.7
B:GPIO5	-	O:CMFLASH1	B0:SPI5_A_MO	I1:DDIG	I0:I2SIN4_DATA0	-	O:DBG_MON_A25
B:GPIO6	-	O:CMFLASH2	B0:SPI5_A_MI	I0:VBUSVALID	I0:I2SIN4_DATA1	-	O:DBG_MON_A26
B:GPIO151	B1:SCL7	O:CMFLASH3	O:SPI5_A_CLK	-	I0:I2SIN4_DATA2	I1:MD32_0_RXD	O:DBG_MON_A27
B:GPIO152	B1:SDA7	-	O:SPI5_A_CSB	-	I0:I2SIN4_DATA3	O:MD32_0_TXD	O:DBG_MON_A28
B:GPIO33	B1:KPROW0	O:I2SIN0_MCK	O:BPI_BUS22	O:CONN_BPI_BUS22	-	-	O:DBG_MON_A29
B:GPIO34	B1:KPROW1	O:I2SIN1_MCK	-	-	-	-	-
B:GPIO94	O:BPI_BUS9	O:CONN_BPI_BUS9	-	O:PWM_3	O:AUD_DAC_26M_CLK	-	O:DBG_MON_A9
B:GPIO95	O:BPI_BUS10	O:CONN_BPI_BUS10	-	-	O:I2SOUT4_DATA0	-	O:DBG_MON_A10
B:GPIO96	O:BPI_BUS11	O:CONN_BPI_BUS11	-	-	O:I2SOUT4_DATA1	-	O:DBG_MON_A11
B:GPIO97	O:BPI_BUS12	O:CONN_BPI_BUS12	B0:SPI6_B_MI	-	O:I2SOUT4_DATA2	-	O:DBG_MON_A12
B:GPIO98	O:BPI_BUS13	O:CONN_BPI_BUS13	O:SPI6_B_CLK	-	O:I2SOUT4_DATA3	-	O:DBG_MON_A13
B:GPIO99	O:BPI_BUS14	O:CONN_BPI_BUS14	O:SPI6_B_CSB	-	O:I2SIN4_BCK	-	O:DBG_MON_A14
B:GPIO100	O:BPI_BUS15	O:CONN_BPI_BUS15	B0:SPI6_B_MO	-	I0:I2SIN4_DATA0	-	O:DBG_MON_A15
B:GPIO101	O:BPI_BUS16	O:CONN_BPI_BUS16	O:CLKM0	-	I0:I2SIN4_DATA1	-	O:DBG_MON_A16
B:GPIO102	O:BPI_BUS17	O:CONN_BPI_BUS17	O:CLKM1	-	I0:I2SIN4_DATA2	-	O:DBG_MON_A17
B:GPIO103	O:BPI_BUS18	O:CONN_BPI_BUS18	O:CLKM2	O:DMIC1_CLK	I0:I2SIN4_DATA3	-	O:DBG_MON_A18
B:GPIO104	O:BPI_BUS19	O:CONN_BPI_BUS19	O:CLKM3	I0:DMIC1_DAT	O:I2SIN4_LRCK	-	O:DBG_MON_A19
B:GPIO153	B1:SCL8	O:I2SIN1_BCK	-	-	O:UDI_TDO_4	I0:MFG_TSFDC_26M	-
B:GPIO154	B1:SDA8	O:I2SIN1_LRCK	-	-	O:UDI_TDO_5	I0:MFG_TSFDC_SCF	-
B:GPIO125	O:CMMCLK2	I0:I2SIN1_DI	-	-	I0:UDI_TDI_6	-	-
B:GPIO117	-	O:I2SOUT1_DO	I1:URXD2	O:CLKM2	I0:UDI_TDI_2	O:PWM_2	-
B:GPIO161	B1:SCL12	O:I2SIN2_BCK	I1:UCTS2	-	O:CMVREF3	I0:MFG_TSFDC_SCK	-
B:GPIO162	B1:SDA12	O:I2SIN2_LRCK	O:URTS2	-	O:CMVREF2	I0:MFG_TSFDC_SDI	-
B:GPIO129	O:CMMCLK6	I0:I2SIN2_DI	-	-	O:CMVREF1	-	-
B:GPIO121	-	O:I2SOUT2_DO	O:UTXD2	O:PWM_2	O:CMVREF0	-	-
B:GPIO78	B0:MSDC2_CLK	O:I2SIN6_0_BCK	B0:TP_GPIO11_AO	I0:CONN_BGF_DSP_L1_JCK	I1:SCP_ITAG0_TCK_VLP	I1:SSPM_ITAG_TCK_VLP	I0:IO_ITAG_TCK
B:GPIO79	B1:MSDC2_CMD	O:I2SIN6_0_LRCK	B0:TP_GPIO12_AO	I1:CONN_BGF_DSP_L1_JMS	I1:SCP_ITAG0_TMS_VLP	I1:SSPM_ITAG_TMS_VLP	B1:IO_ITAG_TMS
B:GPIO80	B1:MSDC2_DAT0	I0:I2SIN6_0_DI	B0:TP_GPIO13_AO	I0:CONN_BGF_DSP_L1_JDI	I1:SCP_ITAG0_TDI_VLP	I1:SSPM_ITAG_TDI_VLP	I1:IO_ITAG_TDI
B:GPIO83	B1:MSDC2_DAT3	O:I2SOUT6_0_DO	O:PWM_VLP	I0:MD_INT3	B0:TP_GPIO4_AO	O:SRCLKENA1	I0:SRCLKENA10
B:GPIO159	B1:SCL11	O:I2SIN0_BCK	O:BPI_BUS16	O:ANT_SEL16	O:CMVREF4	O:MFG_TSFDC_TSSE_L0	-
B:GPIO160	B1:SDA11	O:I2SIN0_LRCK	O:BPI_BUS17	O:ANT_SEL17	O:CMVREF5	O:MFG_TSFDC_RCK_SELB	-
B:GPIO128	O:CMMCLK5	I0:I2SIN0_DI	O:BPI_BUS18	O:ANT_SEL18	O:CMVREF6	-	-
B:GPIO120	O:PWM_1	O:I2SOUT0_DO	O:BPI_BUS19	O:ANT_SEL19	O:CMVREF7	-	-
B:GPIO184	O:ANT_SEL11	O:I2SIN6_0_BCK	I0:PTA_EXT_FREQ	O:BPI_BUS11	O:CMVREF4	O:CLKM0	-
B:GPIO185	O:ANT_SEL12	O:I2SIN6_0_LRCK	I0:PTA_EXT_ACT	O:BPI_BUS12	O:CMVREF5	O:CLKM1	-
B:GPIO186	O:ANT_SEL13	I0:I2SIN6_0_DI	I0:PTA_EXT_PRI	O:BPI_BUS13	O:CMVREF6	O:CLKM2	-
B:GPIO187	O:ANT_SEL14	O:I2SOUT6_0_DO	O:PTA_EXT_WLAN_ACT	O:BPI_BUS14	O:CMVREF7	O:CLKM3	-

1.3.1.2 I2S 应用

表 1-2. I2S 应用

Interface	Direction	Suggestions for I2S IF assignment
I2S0	Input/Output	Reserved, for 3rd patry
I2S1	Input/Output	Reserved, for 3rd patry
I2S2	Input/Output	Reserved, for 3rd patry
I2S4	Input/Output	Ext DSP, SmartPA w/I DSP
I2S6	Input/Output	Reserved, for 3rd patry
MTK proprietary Audio IF	Input*4 Output*4	ADC *4, for Main Mic, Ref Mic, 3rd Mic, 4th Mic, DMIC*4 DAC *4, for receiver, headset L, headset R, ext amp

其他应用，请咨询 MTK ACS 团队以获得最佳 I2S IF 分配。

对于 smartPA 的引脚应用，推荐使用 I2S4。

1.4 联发科 Aurisys 和开放式 DSP

1.4.1 概述

人们对音质增强以增加聆听乐趣的需求日益增加。为了丰富使用移动设备的聆听体验，供应商提供的声音处理解决方案的数量不断增加。

Aurisys 的推出是为了促进 MTK 平台声音处理解决方案的开发和使用。Aurisys 是基于 Android 音频框架构建的框架。它包括用于声音处理和调音的标准化接口、集成的 DSP 声音子系统和软件调试接口。本文将简要介绍 Aurisys 的概念。

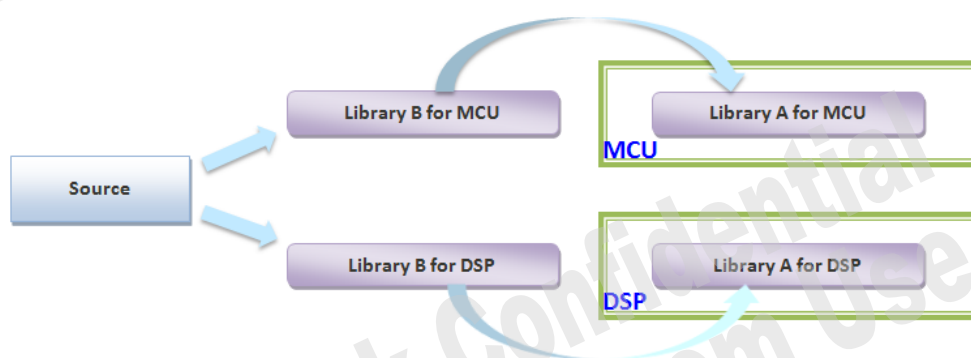


图 1-12. Aurisys 概念

1.4.1.1 Aurisys 结构

如图 1-13 所示，Aurisys 结构包含场景处理程序、库管理器、集成 DSP 框架、调制解调器/音频硬件子系统和标准化软件接口：

- Aurisys Scene Handler (ARSH)
 - Aurisys Scene Handler 是音频系统和声音处理 IP 之间的中间件，是按场景创建的。例如，Aurisys Playback Handler 是用于播放场景中的播放效果。它提供调用相应 IP 的接口并管理 IP。
- Aurisys Library Manager (ARLM)
 - Aurisys 库管理器维护每个 Aurisys 场景处理程序的库信息。它包含声音算法 IP 的列表和状态。
- Aurisys Software Interface (ARSI)
 - ARSI 是 Aurisys Software Interface 的缩写。它用于与声音增强任务进行交互。为了处理声音，提供了一个可在 MTK 平台之间移植的统一接口。还包括在 PC 工具、APMCU 和 DSP 之间解析和传输参数的接口。该接口设计易于使用，同时足够通用，因此可以轻松将新算法添加到现有框架中。ARSI 和 ARSH 之间的区别在于，ARSI 提供了 ARSH 和声音 IP 之间的接口；而 ARSH 提供了 Android Audio HAL 请求处理声音的接口。
- Modem

- 调制解调器是指调制解调器 IC。MTK 内部芯片组包含一个 DSP，用于处理语音增强和语音编解码器。在 Aurisys 结构中，我们禁用 modem 内部的语音增强，但在调制解调器 IC 中保留编解码器，以便我们只需在应用程序端集成的开放 DSP 中添加算法即可。
- Audio HW
 - 指处理器与硬件设备之间传输声音数据的接口。这些设备包括扬声器、麦克风、耳机、蓝牙设备、USB 设备等。

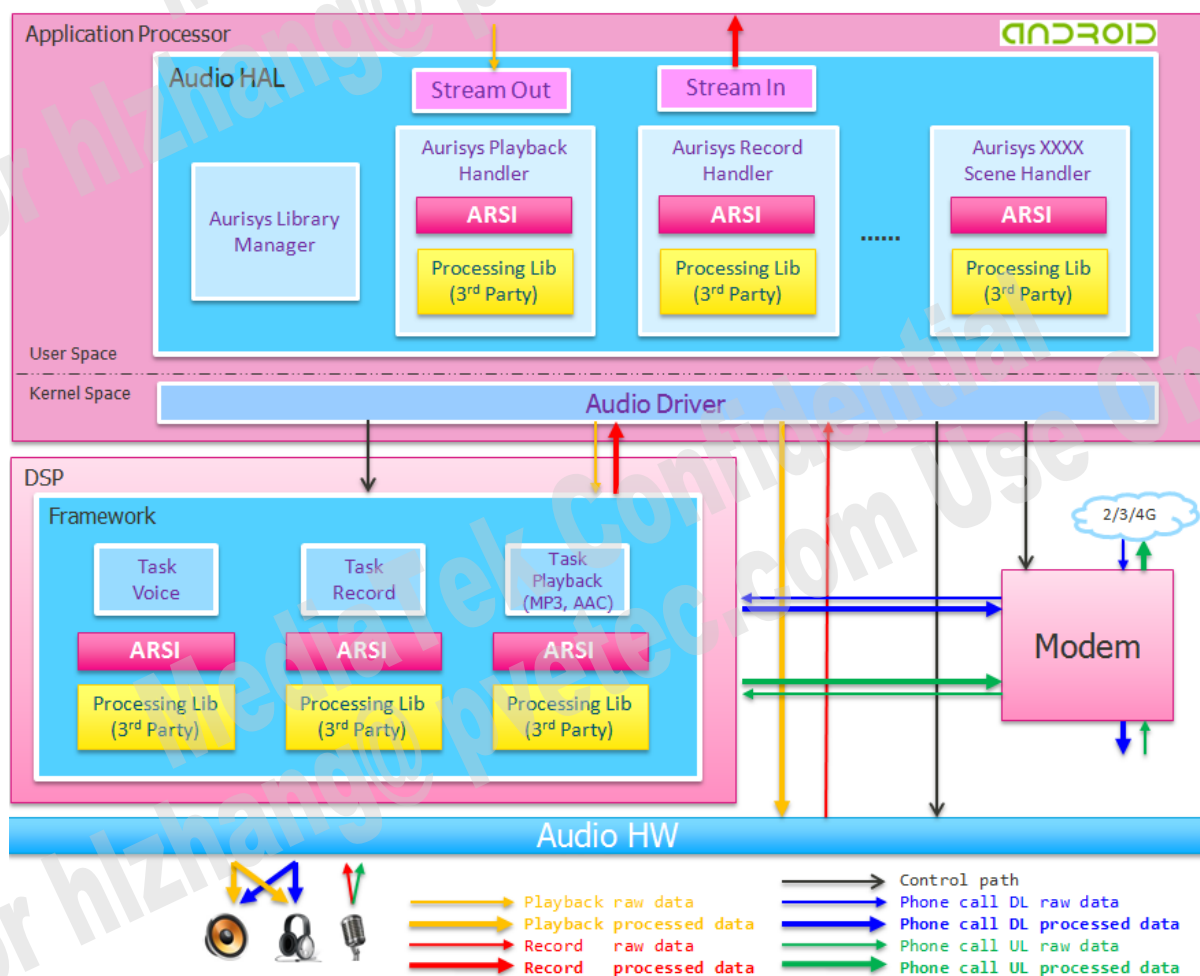


图 1-13. Aurisys 结构

1.4.2 数据路径客制指南

使用 DSP 进行音频处理和输出时，需要在 DSP 中配置每条音频流的混音路由路径和通道配置信息。公共的 DSP 任务配置提供了基本的参考配置，但在实际场景中，需要对各个 DSP 任务进行一些修改。本节主要介绍 ADSP 相关功能定制所涉及的配置文件修改。

1.4.2.1 Kernel DTS 配置

在 DTS 中，与 DSP 音频流相关的属性配置如下：

```
&snd_audio_dsp {
    mtk-dsp-call-final = <0x0 0xe 0x13 0x12 0x18000>;
    mtk-dsp-hfp-client-rx = <0x5 0xffffffff 0x12 0xffffffff 0x20000>;
    mtk-dsp-anc = <0x5 0xffffffff 0x10 0xffffffff 0x20000>;
    mtk-dsp-extendstream1 = <0x5 0xffffffff 0xf 0xffffffff 0x20000>;
    mtk-dsp-extendstream2 = <0x5 0xffffffff 0x11 0xffffffff 0x20000>;
    mtk-dsp-sub-playback = <0x1 0x0 0xffffffff 0xffffffff 0x30000>;
    mtk-dsp-playback0 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback1 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback2 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback3 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback4 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback5 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback6 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback7 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback8 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback9 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback10 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback11 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback12 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback13 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback14 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback15 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-capture-mch = <0x1 0xffffffff 0x18 0xffffffff 0x20000>;
    swdsp-smartpa-process-enable = <0x0>;
};
```

图 1-14. DTS 属性列表

每个 DSP 任务属性节点保存一个五位数组，其中数组顺序代表了该 DSP 任务使用的共享内存的 enable、dl_mem、ul_mem、ref_mem 和大小。在所有属性中，用户需要根据每个 DSP 任务对 memif 的实际使用情况来配置 dl_mem（下行）、ul_mem（上行）、ref_mem（参考信号）。例如，对于 mtk-dsp-sub-playback 任务，使用的 dl 输出 memif 为 DL0，那么对应的 dl_mem 就填充 0x0。大部分任务（比如 playback0）实际上并不使用 hw memif，所以需要配置为 0xffffffff。memif 节点的枚举类型定义在 mt6897-afe-common.h 中。

```
enum {
    MT6991_MEMIF_DL_4CH,
    MT6991_MEMIF_DL_24CH,
    MT6991_MEMIF_VUL0,
    MT6991_MEMIF_VUL1,
    MT6991_MEMIF_VUL2,
    MT6991_MEMIF_VUL3,
    MT6991_MEMIF_VUL4,
    MT6991_MEMIF_VUL5,
    MT6991_MEMIF_VUL6,
    MT6991_MEMIF_VUL7,
    MT6991_MEMIF_VUL8,
    MT6991_MEMIF_VUL9,
    MT6991_MEMIF_VUL10,
    MT6991_MEMIF_VUL24,
    MT6991_MEMIF_VUL25,
    MT6991_MEMIF_VUL26,
    MT6991_MEMIF_VUL_CM0,
    MT6991_MEMIF_VUL_CM1,
    MT6991_MEMIF_VUL_CM2,
    MT6991_MEMIF_ETDM_IN0,
    MT6991_MEMIF_ETDM_IN1,
    MT6991_MEMIF_ETDM_IN2,
    MT6991_MEMIF_ETDM_IN3,
    MT6991_MEMIF_ETDM_IN4,
    MT6991_MEMIF_ETDM_IN6,
    MT6991_MEMIF_HDMI,
    MT6991_MEMIF_NUM,
};
```

图 1-15. memif 定义

需要注意的是，在修改了 dts 中对应的 task 使用的节点后，同时也需要修改 hal 中对应的 task 使用的 pcm。Dsp-playback task 对应修改 audio hal 中 AudioDspStreamManager.cpp 中的 openPlaybackTask 函数中的 mPlaybackUlindex 等变量，选用对应 dl 的 playback stream，其中 dsp 输出的 channel 数量等也在该函数处修改，如图 1-16 所示。

```
.../* .get.card.index and pcm.index*/
.../* .todo : pcm should move to platform */
...mStreamCardIndex = AudioALSADeviceParser::getInstance()->GetCardIndexByString(keypcmPlayback11);
...mDspIndex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmPlaybackDsp);

...mPlaybackUlindex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmCapture6);
...mPlaybackIvindex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmCapture4);
...mPlaybackDlindex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmPlayback11);
```

图 1-16. 音频 HAL PCM Config 配置

DSP 下的各个 task 的架构如下：

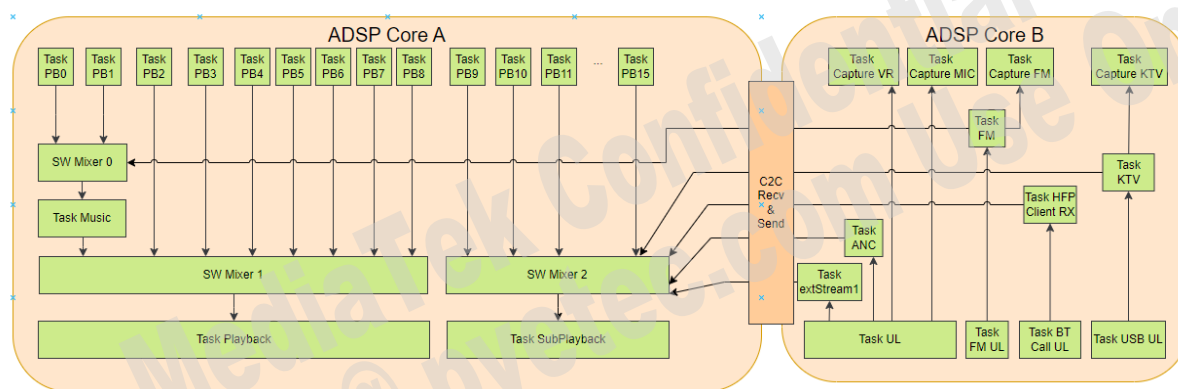


图 1-17. ADSP 架构

1.4.2.2 音频路径配置 (AFE HW)

在通过 audio dsp pcm stream 进行播放和录音之前，需要配置对应的 AFE hardware 通路，其配置过程遵循 tinyalsa 的标准流程，通过 tinymix 配置相应的 kcontrol 通路，并 open 和 start 相应的 hw 通路。图 1-18 为配置 DL3->ADDA 的播放通路。

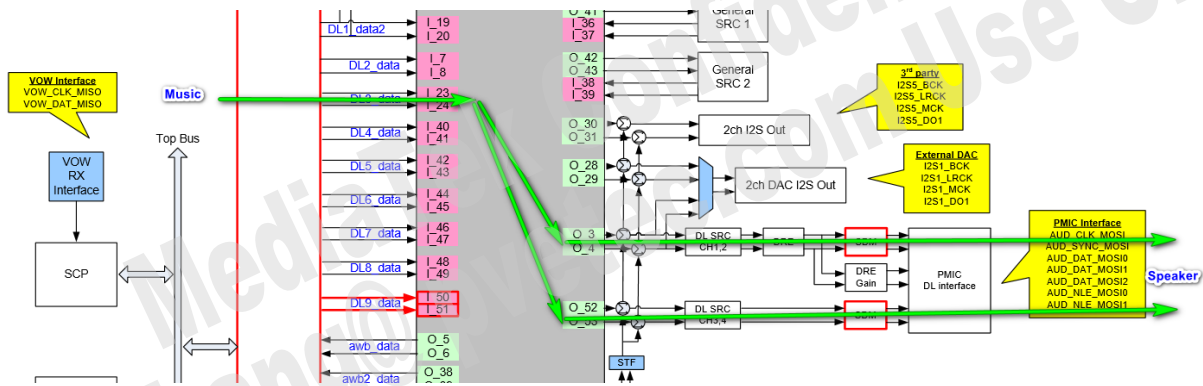


图 1-18. 播放音乐的音频路径

在 audio_device.xml 中相应的配置如下：

```
<path name="PLAYBACK3_TO_ADDA_DL" value="turnon">
  <kctl name="ADDA_DL_CH1 DL3_CH1" value="1" />
  <kctl name="ADDA_DL_CH2 DL3_CH2" value="1" />
  <kctl name="ADDA_DL_CH3 DL3_CH1" value="1" />
  <kctl name="ADDA_DL_CH4 DL3_CH2" value="1" />
</path>
<path name="PLAYBACK3_TO_ADDA_DL" value="turnoff">
  <kctl name="ADDA_DL_CH1 DL3_CH1" value="0" />
  <kctl name="ADDA_DL_CH2 DL3_CH2" value="0" />
  <kctl name="ADDA_DL_CH3 DL3_CH1" value="0" />
  <kctl name="ADDA_DL_CH4 DL3_CH2" value="0" />
</path>
<path name="headphone_output" value="turnon">
  <kctl name="DAC In Mux" value="Normal Path" />
  <kctl name="HPL Mux" value="Audio Playback" />
  <kctl name="HPR Mux" value="Audio Playback" />
</path>
<path name="headphone_output" value="turnoff">
  <kctl name="HPL Mux" value="Open" />
  <kctl name="HPR Mux" value="Open" />
</path>
```

图 1-19. 将 Playback_3 传输到 PMIC

1.4.2.3 音频路径配置 (INT ADSP)

Auto adsp 中会存在多个 task (bus out)，且每个 task 根据需求及实际场景不同，需要配置不同的算法处理参数，例如 channel in，channel out 及是否 bypass 算法等。同时，一般需要将各路 bus 的 channel 映射到最终输出的 TDM 格式中对应的 channel 位置，即需要配置 channel map 信息，故提供 audio_dsp_config.xml 配置修改 auto dsp task 的各项属性及 mix path。一般的 adsp 中的 task 架构如图：

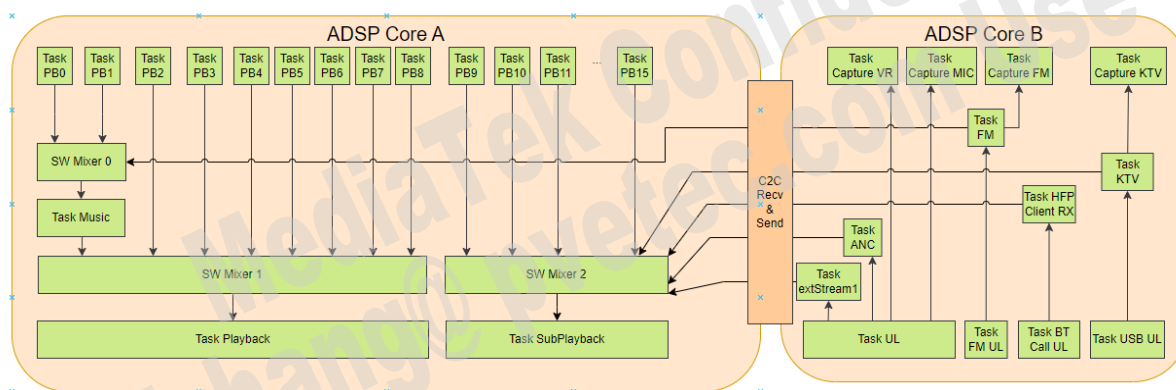


图 1-20. ADSP 架构

1.4.2.3.1 XML 配置指令

配置 XML 路径：

- xml 在设备上的 path 如下：vendor/etc/audio_dsp_config.xml
- 在 code 中 path 如下：device/<project_name>/ audio_dsp_config.xml

1.4.2.3.2 mixer_target 和 mixer_source

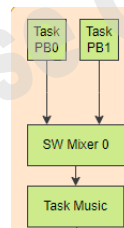
XML 中将每个 dsp task 根据其在 dsp 中的 mixer 的位置分为了 sw_mixer_target 和 sw_mixer_source。对于 Task Music，其即是 sw_mixer_target，也是 sw_mixer_source。

- sw_mixer_target
对应于 sw_mixer 的输出端，如 Task Music, Task Playback, Task SubPlayback
- sw_mixer_source
对应于 sw_mixer 的输入端，如 Task PB0 (name="sw_mixer_playback0")

task 作为 target 或者 source 是不可配置的，其角色是预定好的，但是可以通过 XML 配置 sw_mixer_source mix 到期望的 target 中。对应的修改方法是在 XML 中，将对应 name 的 sw_mixer_source 节点写到对应的 sw_mixer_target 的节点下，如下例子表示 task PB0 (sw_mixer_playback0) 将会 mix 到 task music(sw_mixer_music)中。

Ex1:

```
<sw_mixer_target name="sw_mixer_music" ch_in="2" ch_out="16"
aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
      ch_map="0x5555,0xaaaa"/>
  </sw_mixer_source>
</sw_mixer_target>
```



目前可以配置的 sw_mixer_source 有: sw_mixer_playback0 ~ sw_mixer_playback15, sw_mixer_FM_ADSP, sw_mixer_hfp_client_rx, sw_mixer_anc, sw_mixer_extstream1, sw_mixer_extstream2, sw_mixer_ktv.

1.4.2.3.3 任务属性

在每个 dsp task, 由于场景需要, 会配置不同的算法输入输出 channel 及 channel map 信息。在 XML 中, 可以为每个 task 分别配置下列属性:

- for sw_mixer_target
 - ch_in: Task 的算法输入 channel
 - ch_out: Task 的算法输出 channel
 - aurisys_on: 是否开启 Task 算法, 仅在 ch_in = ch_out 时可配置成 0

例如, 在下面例子中, task music 的算法输入 channel 是 2, 算法处理后的输出 channel 是 16, 算法非 bypass.

```
<sw_mixer_target name="sw_mixer_music" ch_in="2" ch_out="16" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
      ch_map="0x5555,0xaaaa"/>
  </sw_mixer_source>
</sw_mixer_target>
```

- for sw_mixer_source, 每个 profile 有如下属性
 - ch_in: Task 的算法输入 channel
 - ch_out: Task 的算法输出 channel
 - aurisys_on: 是否开启 Task 算法, 仅对 pb0~pb15 在 ch_in = ch_out 时配置成 0 可生效
 - ch_map: source 在向 sw_mixer_target 映射时的 channel map 信息, 是一个数组, 数组中的 index 表示 source 的 channel index, 对应的值是一个 16bit 的 value, 每一个 bit 表示是否向 target 对应 channel 映射。如 ch_map="0x5555,0xaaaa", ch_map[0] = 0x5555, 表示 source 的 channel 1 映射数值为 0x5555, 对应 2 进制为 0101010101010101, 则向 target 的 1,3,5,7,9,11,13,15 channel 映射。ch_map[1] = 0xaaaa, 对应二进制位 1010101010101010, source 的 channel 2 向 target 的 2,4,6,8,10,12,14,16 映射。

对 source task, 一般其输入 channel 是由数据原始端给出的, 如 audioflinger 送下来, 可能会是任意 channel 数量, 所以在 source task 中, XML 可配置支持不同的 ch_in 的多种 profile, 可以根据不同的 ch_in 选择不同的 ch_out, ch_map 等信息。例如, 如下 task pb14 具有两个 profile, 当进入 adsp 给到 task pb14 的 channel 是 2ch, 则使用第一个 profile, 如果是 12ch, 则使用第二个。

```
<sw_mixer_source name="sw_mixer_playback14">
  <profile ch_in="2" ch_out="2" aurisys_on="0"
    ch_map="0x5555,0xaaaa"/>
  <profile ch_in="12" ch_out="12" aurisys_on="0">
```

```
ch_map="0x1,0x2,0x4,0x8,0x10,0x20,0x40,0x80,0x100,0x200,0x400,0x800,0x1000,0x2000,0x4000,0x8000"/>
</sw_mixer_source>
```

对于 music task，由于其即是 source，也是 target，则会取其 target 属性中的 ch_in 属性，在 source 中找到对应的 profile 作为其配置。

1.4.2.3.4 配置示例

如下是一个 task pb0 配置示例：

```
<sw_mixer_target name="sw_mixer_music" ch_in="8" ch_out="12" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
      ch_map="0x4,0x8"/>
    <profile ch_in="4" ch_out="4" aurisys_on="0"
      ch_map="0x1,0x2,0x4,0x8 "/>
  </sw_mixer_source>
</sw_mixer_target>

<sw_mixer_target name="sw_mixer_playback" ch_in="16" ch_out="16" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_music">
    <profile ch_in="8" ch_out="12" aurisys_on="1"
      ch_map="0x1,0x2,0x4,0x8,0x10,0x20,0x40,0x80,0x100,0x200,0x400,0x800
"/>
  </sw_mixer_source>
</sw_mixer_target>
```

对应的数据通路流程如图 1-21：

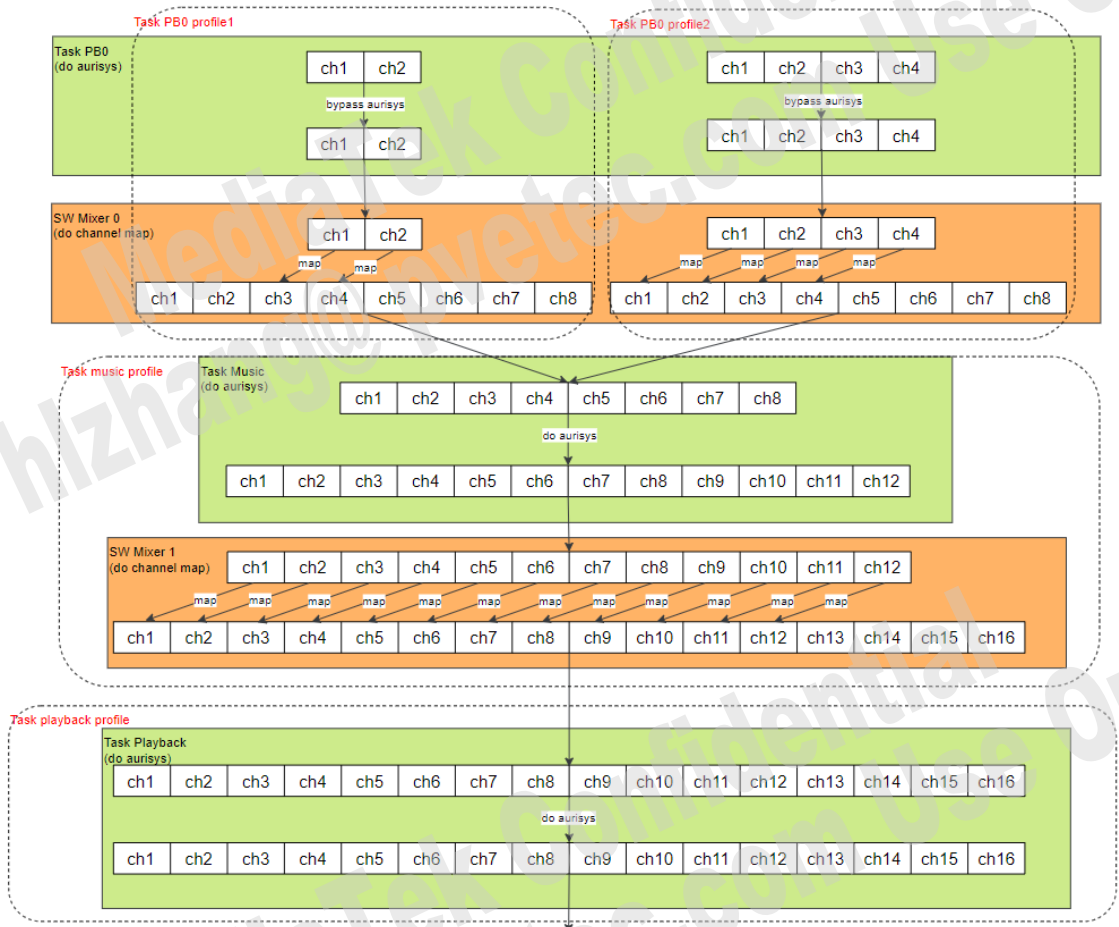


图 1-21. DSP 任务配置

1.4.3 总结

本节主要介绍 Aurisys 音效框架，方便用户在适当位置整合所需算法。同时，面对 bus 场景音频通路的客制化需求，介绍了 ADSP 中音频 mix 和 channel map 的配置方法。

除了上述功能外，还可实现其他应用。但考虑到 DSP 资源有限，并发情况应谨慎设计。请访问联发科网站和客户支持网站 (MediaTek On-Line, MOL) 以获取更多信息。

1.5 Audio 各模块 Debug 方法介绍

1.5.1 Audio Framework Debug

打开工程模式 Dump audio framework 步骤如下（在 android 端 shell 操作）：

- 1) adb shell am start -n com.mediatek.engineermode/.EngineerMode
- 2) 向右滑动到“Hardware Testing”，点击“Audio”
- 3) 进入“Audio Logger”
- 4) 将“Output”、“AudioMixer”全部勾选(上行则选 Input)
- 5) 复现问题
- 6) 回到工程模式，取消相应勾选
- 7) dump 的数据路径：/data/debuglogger/audio_dump

动态开启 Audio framework debug log

- 1) adb root
- 2) adb shell setprop vendor.af.track.log 4
- 3) adb shell setprop vendor.af.audioflinger.log 4
- 4) adb shell setprop vendor.af.policy.debug 4
- 5) adb shell setprop log.tag.APM_AudioPolicyManager V
- 6) adb shell pkill audioserver

1.5.2 Audio HAL Debug

Dump Audio HAL 命令如下（在 android 端 shell 操作）：

下行播放：

- 1) adb root
- 2) adb shell “AudioSetParam SetDumpAudioStreamOut=268433409”
- 3) 复现问题
- 4) adb shell “AudioSetParam SetDumpAudioStreamOut=0”

上行录音：

- 1) adb root
- 2) adb shell “AudioSetParam SetDumpAudioStreamIn=1”
- 3) 复现问题
- 4) adb shell “AudioSetParam SetDumpAudioStreamIn=0”

Dump 路径：/data/vendor/audihal/audio_dump，除去 audio HAL dump，还有 adsp task dump.

```
auto8676pl_64_bsp_vm:/data/vendor/audiohal/audio_dump # ls
TaskPlayback0_2_in_20250612-054725.pcm  adsp_Playback_0_out_20250612-054725.pcm
TaskPlayback0_2_out_20250612-054725.pcm  hal_playbackhandler_BusDsp_flag2_out_pid6420.tid6693_20250612-054725.wav
adsp_Music_1_in_20250612-054725.wav      hal_playbackhandler_BusNormal_flag2_out_pid6420.tid6494_20250612-054723.wav
adsp_Music_1_out_20250612-054725.wav     hal_streamout_flag2_out_pid6420.tid6494_20250612-054723.wav
adsp_Playback_0_in_20250612-054725.pcm   hal_streamout_flag2_out_pid6420.tid6693_20250612-054725.wav
```

动态开启 Audio HAL debug log（在 android 端 shell 操作）

```
streamout property: vendor.streamout.log
streamin property: vendor.streamin.log
```

下行播放：

```
adb root
adb shell setprop vendor.streamout.log xxx
adb shell pkill audioserver
```

上行录音：

```
adb root
adb shell setprop vendor.streamin.log xxx
adb shell pkill audioserver
```

xxx 选自图 1-22。

```
202 // enum for MTK_AUDIO_DYNAMIC_LOG
203 enum {
204     AUD_LOG_DEBUG_EN      = 0x1 << 0,
205     AUD_LATENCY_MONITOR   = 0x1 << 1,
206     AUD_LATENCY_MONITOR_ALL = 0x1 << 2,
207     AUD_OUT_OP_LOG        = 0x1 << 3,
208     AUD_OUT_WRITE_LOG     = 0x1 << 4,
209     AUD_DSP_STREAM        = 0x1 << 5,
210 };
```

图 1-22. Audio HAL enum 值

举个例子，如果需要查看 audio 下行 write 节奏，可以在 android 端 shell 输入如下命令。

```
adb root
adb shell setprop vendor.streamout.log 16
adb shell pkill audioserver
```

```
06-12 05:35:55.104 6420 6494 D AudioALSAPlaybackHandlerBusNormal: +write(), flag = 2,
buffer = 0xb400007c7b49d000, bytes = 2048
06-12 05:35:55.104 6420 6494 D AudioALSAPlaybackHandlerBusNormal: -write(), flag = 2,
buffer = 0xb400007c7b49d000, bytes = 2048
06-12 05:35:55.104 6420 6494 D AudioALSAPlaybackHandlerBusNormal: +write(), flag = 2,
buffer = 0xb400007c7b49d000, bytes = 2048
06-12 05:35:55.105 6420 6494 D AudioALSAPlaybackHandlerBusNormal: -write(), flag = 2,
buffer = 0xb400007c7b49d000, bytes = 2048
06-12 05:35:55.107 6420 6494 D AudioALSAPlaybackHandlerBusNormal: +write(), flag = 2,
buffer = 0xb400007c7b49d000, bytes = 2048
```

```
06-12 05:35:55.111 6420 6494 D AudioALSAPlaybackHandlerBusNormal: -write(), flag = 2,
buffer = 0xb400007c7b49d000, bytes = 2048
```

Playback/Capture log tag:

```
AudioALSASStream|AudioALSAPLAYBACK|AudioALSACapture|AudioDspStreamManager
```

HFP Client log tag:

```
AudioHfpDLTaskSink|AudioHfpULTaskSink|AudioBTCVSDControl|AudioHfpDLTaskSource|AudioHfpULTask
Source|AudioHfpController
```

Dump HFP client 命令如下:

```
adb root
adb shell vendor.streamhfp.pcm.dump 1
adb shell pkill audioserver
```

1.5.3 ADSP Debug

ADSP Dump 请参考 Audio HAL Dump。

查看 ADSP mobile log:

Path: /data/debuglogger/mobilelog/adsp_log_xxx

ADSP task 可以从 TASK_DEBUG_INFO 修改为 TASK_DEBUG_VERBOSE, 参考图 1-23。

再 build audio_dsp.img 烧入平台获取详细 log 打印。

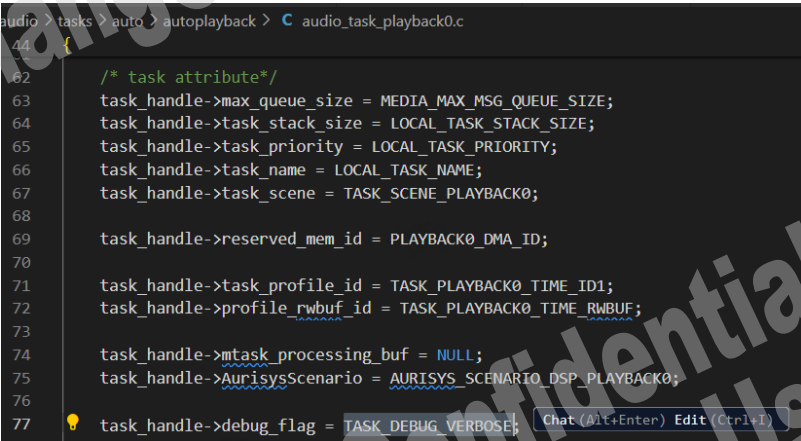


图 1-23. ADSP task log 配置

更多详细 debug 方法请参考: Aurisys_exe_V1.xxx.xxx\DOC

1.5.4 Audio 虚拟化 Debug

虚拟化主要涉及性能问题分析。分为：VAFE 和 VADSP。

VAFE Case:

数据不通过 ADSP。关注两个线程 1. Writer/Reader 线程写/读数据; 2. vafe timer 线程更新 HW DMA Buffer Rp/Wp。

Step 1: 在 android trace log 搜索断音发生的 trace marker 关键字 `afe_audio_xrun`，搜到后，表示这份 trace 有抓到断音位置。

Step 2: 把 trace log 拖到 perfetto，转成图形化界面进行查看。<https://ui.perfetto.dev/>

Step 3: 查找 `audio_xrun` trace marker 对应的线程 writer 或者 reader 线程，这个线程由于开了 mbrain 和 binder 通信，被 binder 修改名称为 `binderxxx`。参考图 1-24。

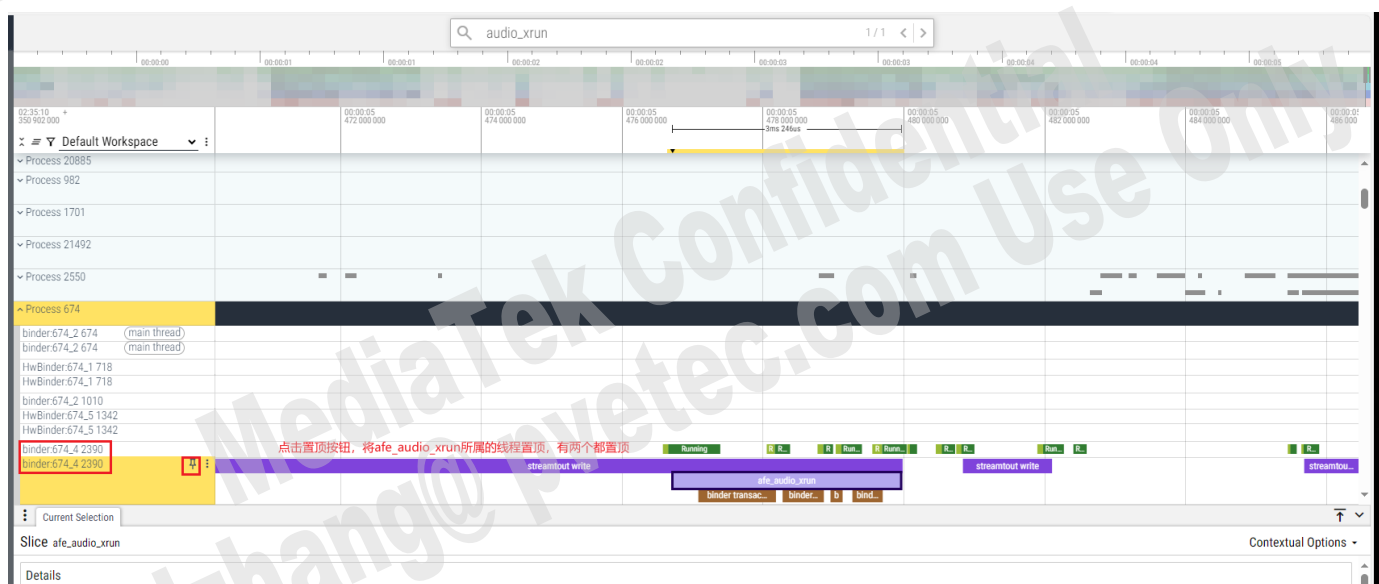


图 1-24. VAFE binder 线程

Step 4: 找到 `vae_timer` 线程，进行置顶操作。参考图 1-25。

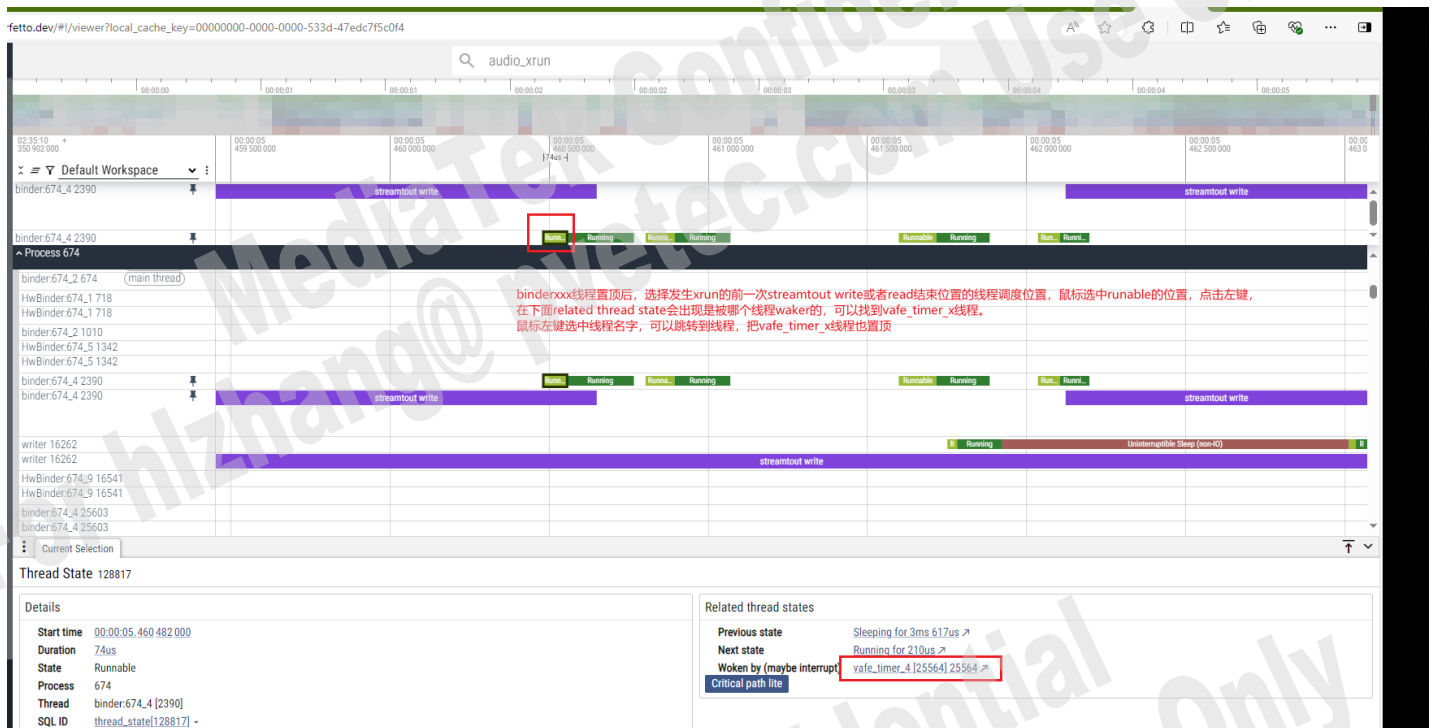


图 1-25. VAFE timer

Step 5: 定位问题点，对比前面正常写的时候这两个线程的状态，和异常情况下的线程状态。参考图 1-26。

初步定位问题：

1. vafe_timer_x 线程调度出现延迟，正常情况下应该每 5ms 调度一次，但在断音时却延迟到了 16ms 才进行调度。
2. vafe_timer_x 正常时间到达，但是长时间处于 runnable 状态，表示线程准备好运行，但 CPU 目前没有空闲。可以查看这个时间段否有长时间运行的线程，找到阻塞 vafe_timer_x 的线程。
3. vafe_timer_x 运行正常，binderxx 线程长时间处于 runnable 状态，表示 binderxx 线程被阻塞，可以查看这个时间段是否有其他线程长时间占用 CPU，找到阻塞 binderxx 的线程。

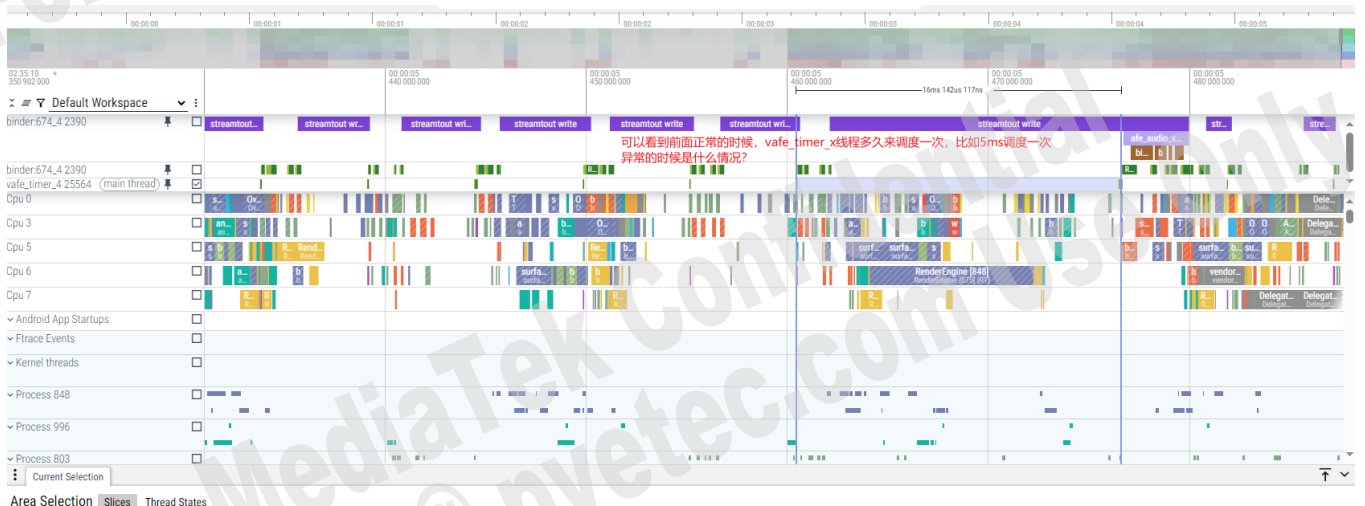


图 1-26. VAFE trace 分析

VADSP case:

Android 端关注两个线程: 1. writer/reader 线程; 2. elapse_task 线程

Yocto 端关注 5 个线程: 1. ioreq_worker; 2. VM0-ioeventfd-0; 3. vhost-adsp; 4 dsp_rcv_thread; 5, dsp_send_thread

一次写数据的流程是 writer → wait; vhost-adsp → elapse_task → writer wake up → ioreq_worker → VM0-ioeventfd-0 → vhost-adsp → dsp_send_thread。

一次读数据的流程是 reader → wait; dsp_rcv_thread → vhost-adsp → elapse_task → reader wake up → ioreq_worker → VM0-ioeventfd-0 → vhost-adsp → dsp_send_thread。

Step 1: 在 android trace log 搜索断音发生的 trace marker 关键字 audio_xrun, 搜到后, 表示这份 trace 有抓到断音位置。

Step 2: 把 android 和 yocto trace log 分别拖到 perfetto, 转成图形化界面进行查看。 <https://ui.perfetto.dev/>

Step 3: 在 android trace 查找 audio_xrun, trace marker 对应的线程 writer 或者 reader 线程, 这个线程由于开了 mbrain 和 binder 通信, 被 binder 修改名称为 binderxxx, 同时也找到 elapse_task 线程。参考图 1-27。

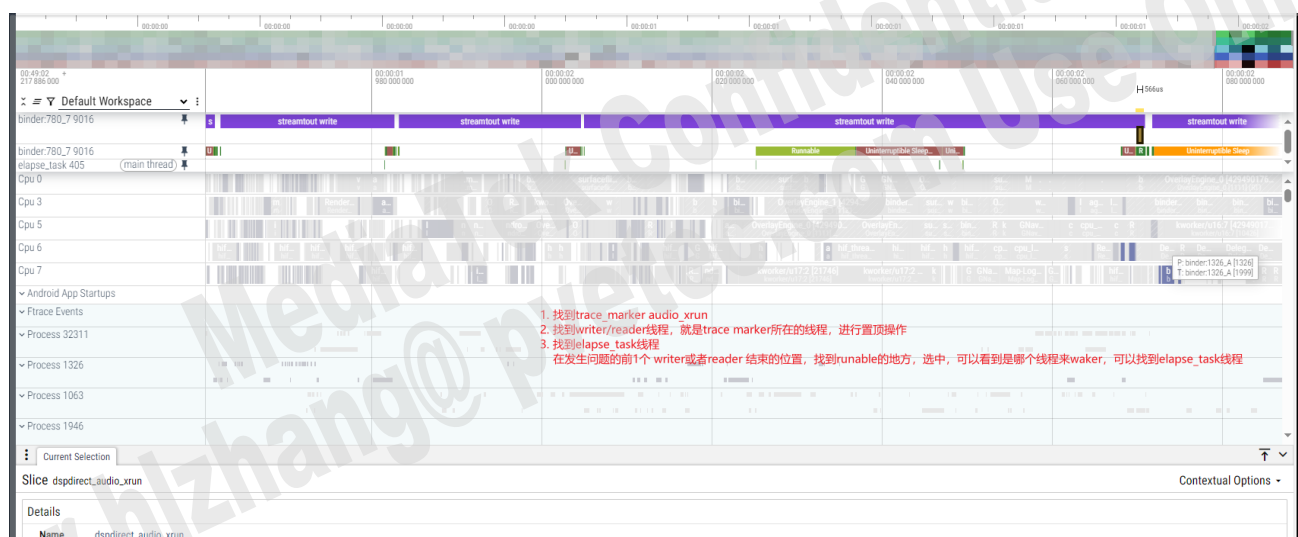


图 1-27. VADSP android trace

Step 4: 要判断问题是发生在 Android 端还是 Yocto 端, 可以根据 elapse_task 的调用频率进行分析。如果在最后一次异常时 elapse_task 的调用出现延迟, 那么问题可能出现在 Yocto 端; 如果 elapse_task 按时调用而问题仍然存在, 则问题可能在 Android 端。

如果 elapse_task 的调用出现延迟, 可以进一步查看在 elapse_task 应该到达的时间与实际调度的时间之间 CPU 的使用情况。具体来说:

- 如果发现有线程在这段时间内一直占用 CPU, 则可以判断该线程可能进行了长时间调用并禁用了中断, 从而导致 adsp 事件中中断响应延迟。
- 如果没有发现这样的线程, 则可以判断问题可能发生在 Yocto 端。Step 5: 分析 Yocto trace log。把需要关注的线程都置顶查看。参考图 1-28。

先查看 vhost-adsp 调度情况，正常情况下调度时间间隔是规律的，看能否找到异常位置。

根据调用顺序，在异常位置查看是否线程有异常，比如线程长时间处于 runnable 状态，长时间 running，长时间处于 sleep 状态。

找到异常线程，结合这个时间的 CPU 调度情况，就可以找到线程被谁阻塞了。



图 1-28. VADSP yocto trace

1.5.5 Tbox Audio Debug

Tbox 通话常见声音异常问题，需要提供 tbox mobilelog, yocto sos mobilelog, Modem log 以及 VM dump 文件。

tbox mobilelog 在 tbox shell 的 /data/debuglogger/ 目录下，默认有开启；

yocto sos mobilelog 在 yocto shell 的 /data/debuglogger/ 目录下，默认有开启；

Modem log 需要手动开启，在 tbox shell 输入如下命令：

```
emdlogger_ctrl 6
```

在 /data/debuglogger/mdlog1 下会有 modem log；

VM dump 需要手动开启，通话测试前在 tbox shell 输入如下命令：

```
amixer -D mtk_phoncall cset name='Vmlog_on' 1
```

通话结束后 dump 文件在 /var/log/audio_dump/xxxx.vm，同时也会生成通话的 EPL 文件和 ADSP dump 文件：

/data/vendor/audiohal/audio_dump/下 xxx.pcm 和 xxxx.bin。

注：

- xxx_dl_in.pcm 表示 Modem 下行传输到 ADSP 算法处理前的数据。
- xxx_dl_out.pcm 表示 Modem 下行经过 ADSP 算法处理后的数据。

- xxx_ul_in.pcm 表示 Modem 上行传输到 ADSP 算法处理前的数据。
- xxx_ul_out.pcm 表示 Modem 上行经过 ADSP 算法处理后的数据。
- VM 文件需要提供给 MTK 才能解析，解析后可以看到整个上行，下行的语音数据情况。

1.6 附录

1.6.1 MTK MOL

MTK 提供了一个名为 MTK Online 的论坛，用于分享常见问题、电子课程和重要公告：<http://online.mediatek.com>。

您可以输入关键字来搜索相关的常见问题和电子课程；可以浏览文档树以进入相关的音频常见问题和电子课程。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。